CL-NTRU-MS-IRS: A Certificateless Two-Round Multi-Signature Scheme on NTRU Lattices with Iterative Rejection Sampling and Vector Aggregation

Technical Construction Document

March 17, 2026

Abstract

Existing lattice-based multi-signature schemes face a fundamental key-management dilemma: PKI-based constructions require costly certificate infrastructure, while identity-based alternatives suffer from *key escrow*—the Key Generation Center (KGC) holds every user's complete secret key. Certificateless cryptography resolves both issues by splitting the signing key into a KGC-issued partial key and a user-chosen secret value.

This paper introduces **CL-NTRU-MS-IRS**, a certificateless lattice-based multi-signature scheme on NTRU lattices that, to our knowledge, is the first to employ iterative rejection sampling (IRS). The construction rests on three pillars: (i) a *dual-source key architecture* where the effective signing key $\tilde{\mathbf{s}}_i = \mathbf{d}_i + \mathbf{x}_i$ fuses a KGC-issued partial key $\mathbf{d}_i$ with a user-chosen secret $\mathbf{x}_i$; (ii) a *two-round commit–reveal protocol* with epoch-scoped validity binding—the challenge hash incorporates the current epoch and revocation accumulator digest, so that signatures produced in one epoch cannot pass verification under a different epoch's revocation state; and (iii) *iterative rejection sampling* that processes each non-zero challenge coefficient independently, raising the effective Gaussian ratio from $\alpha = r/\|\tilde{\mathbf{s}}c\|$ to $\alpha = r/\|\tilde{\mathbf{s}}\|$. The N individual signature vectors are summed into a single short vector $\mathbf{z} \in R^2$ (full vector aggregation), accompanied by per-signer IRS sign-decision records of $N\kappa$ bits that the verifier uses to reconstruct the per-signer ternary challenge polynomials.

In the random oracle model, we provide security reductions for two complementary adversary classes—a Type I adversary (external attacker with public key replacement capability) and a Type II adversary (malicious KGC with unrestricted partial key generation)—both targeting the Ring-SIS problem on NTRU lattices. For $N = 10$ signers at the 128-bit security level, the multi-signature size is approximately 914 bytes.

Keywords: Certificateless multi-signatures, lattice-based cryptography, NTRU, Ring-SIS, iterative rejection sampling, post-quantum cryptography.

1 Introduction

A multi-signature scheme allows a group of N signers to collaboratively produce a single compact signature on a common message. Such schemes are increasingly important in blockchain consensus protocols [21], wireless medical sensor networks [8], and other distributed settings where multi-party accountability must be achieved without proportional growth in verification data. Deploying multi-signatures at scale, however, confronts a persistent dilemma in key management.

On one hand, PKI-based schemes require a Certificate Authority to bind each public key to its owner through digital certificates, incurring escalating overhead as the user base grows. On the other hand, identity-based cryptography (IBC), introduced by Shamir [30], allows a user's

identity string to serve directly as a public key, eliminating certificates altogether—but at the cost of *key escrow*: the KGC derives every user's full secret key from a master secret and can therefore impersonate any signer.

Certificateless cryptography (CLC), proposed by Al-Riyami and Paterson [2], resolves this tension by splitting each user's signing key into two independently generated components: a *partial key* issued by the KGC (bound to the user's identity) and a *secret value* chosen autonomously by the user. Neither party possesses the complete signing key, thereby eliminating both certificate management and key escrow.

Orthogonally, the rapid advance of quantum computing renders classical cryptographic assumptions insecure [25], positioning lattice-based cryptography—with its worst-case hardness guarantees [1, 24] and apparent quantum resistance—as the leading post-quantum paradigm. Within this paradigm, however, existing lattice-based multi-signature schemes remain constrained by the inefficiency of classical rejection sampling, which demands large Gaussian parameters and inflates signature sizes. Gärtner's [14] recent iterative rejection sampling (IRS) technique offers a potential breakthrough, yet it has so far been applied exclusively to single-signer schemes and never within a certificateless framework.

1.1 Related Work

Prior work can be organized along two dimensions: key management model and rejection sampling technique.

Key management. The majority of lattice-based multi-signature schemes operate in the PKI model [3, 20, 11, 22, 16], inheriting the certificate management burden. Chen et al. [7, 8] constructed identity-based multi-signatures (IBMS and IBMS-pka) on NTRU lattices, eliminating certificates at the cost of key escrow. The same authors proposed a certificateless multi-signature (CLMS) on NTRU lattices [9], but that construction relies on an interactive broadcast protocol with substantial communication overhead and limited signature compactness.

Rejection sampling. BLISS [5] demonstrated the advantage of bimodal Gaussian sampling over Lyubashevsky's unimodal approach [19]. HAETAE [15] extended bimodal sampling to hyperballs. Most recently, Gärtner [14] introduced iterative rejection sampling with carefully designed acceptance function pairs, achieving rejection rates dramatically lower than all prior methods and enabling Fiat-Shamir-type signature sizes competitive with hash-and-sign schemes such as Falcon [13]. To date, this technique has not been applied to any multi-signature construction.

1.2 Contributions

This work makes the following contributions:

1. **Novel combination of IRS and certificateless cryptography.** To our knowledge, CL-NTRU-MS-IRS is the first lattice-based multi-signature scheme to integrate iterative rejection sampling [14] within a certificateless key management framework.
2. **Dual-source key architecture.** Each signer's effective signing key $\tilde{\mathbf{s}}_i = \mathbf{d}_i + \mathbf{x}_i$ fuses a KGC-issued partial key $\mathbf{d}_i$ (identity-bound) with a user-chosen secret $\mathbf{x}_i$, simultaneously eliminating certificate management and key escrow.
3. **Signature compactness via IRS.** By processing each non-zero challenge coefficient independently, the effective Gaussian ratio α is raised by a factor of $\sqrt{\kappa}$ compared with classical methods, yielding multi-signatures of approximately 914 bytes for $N = 10$ signers at the 128-bit security level—significantly smaller than all prior lattice-based multi-signature schemes.

4. **Two-round protocol with full vector aggregation.** The protocol requires only two broadcast rounds. The N individual signature vectors are summed into a single vector $\mathbf{z} \in R^2$; only $N\kappa$ bits of per-signer IRS sign-decision metadata are transmitted alongside to support verification.
5. **Security reductions for two adversary classes.** In the random oracle model, we construct reductions toward Ring-SIS for a Type I adversary (external attacker with public key replacement) and a Type II adversary (malicious KGC with unrestricted partial key oracle). The Type II reduction is complete; the Type I reduction is complete under the natural restriction that no signer with a replaced (and unknown-secret) public key appears in the forgery set. We explicitly identify and discuss the residual gap for the general Type I case.
6. **Epoch-scoped validity binding.** The challenge hash incorporates the current epoch and revocation accumulator digest, ensuring that signatures cannot migrate across epoch boundaries. This provides revocation-like functionality *without key renewal*: unrevoked signers retain their original keys across all epochs.

1.3 Organization

The paper is structured as follows. Section 2 establishes notation and reviews the mathematical tools—lattice definitions, discrete Gaussians, iterative rejection sampling, and ring-based trapdoor generation—that underpin the construction. Section 3 defines the scheme syntax and formalizes the dual-adversary security model with epoch-scoped revocation. Section 4 presents the complete construction, including the two-round protocol, IRS algorithm, aggregation strategy, and verification procedure. Section 5 proves correctness through three sub-propositions addressing algebraic consistency, norm bounds, and termination. Section 6 provides the security analysis: Type I and Type II game-sequence proofs, supporting lemmas on distribution independence and sign-bit simulatability, and a discussion of reduction tightness. Section 7 evaluates concrete parameters, signature sizes, and computational costs. Section 8 summarizes and identifies directions for future work.

2 Preliminaries

We collect the notation, definitions, and technical tools used throughout the paper. Sections 2.1–2.3 cover standard lattice-theoretic material. Section 2.4 presents the iterative rejection sampling framework of Gärtner [14]—the central engine behind our scheme's compact signatures. Section 2.5 reviews the ring-based trapdoor generation algorithm that enables the KGC to issue partial keys.

2.1 Notation

Let n be a power of two and q a prime with $q \equiv 1 \pmod{2n}$. We work in the polynomial ring $R = \mathbb{Z}[x]/(x^n + 1)$ and its quotient $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. An element $f \in R$ is identified with its coefficient vector $[f_0, \ldots, f_{n-1}] \in \mathbb{Z}^n$. Vectors over R^k are associated with nk-dimensional integer vectors. The Euclidean, ℓ_∞, and ℓ_1 norms are denoted $\|\cdot\|$, $\|\cdot\|_\infty$, and $\|\cdot\|_1$ respectively.

Bold uppercase letters denote matrices, bold lowercase letters denote vectors, and standard-weight letters denote ring elements or scalars. We write $\mathbf{j} = [1, 0, \ldots, 0]^T$ for the first unit vector. The notation $x \xleftarrow{\$} S$ means x is sampled uniformly from the set S.

2.2 NTRU Lattice and the Ring-SIS Problem

Definition 2.1 (NTRU Lattice)**.** Given $h \in R_q$, the NTRU lattice is $\Lambda_{h,q} = \{(u, v) \in R^2 : u + h \cdot v \equiv 0 \pmod{q}\}$.

Definition 2.2 (Ring-SIS on NTRU Lattices)**.** Given $h \in R_q$ and a bound $\beta > 0$, the $\mathrm{RSIS}^{\mathrm{NTRU}}_{q,\beta}$ problem asks to find a non-zero vector $(x_1, x_2) \in R^2$ such that $x_1 + h \cdot x_2 \equiv 0 \pmod q$ and $0 < \|(x_1, x_2)\| \leq \beta$.

2.3 Discrete Gaussian Distributions

Definition 2.3 (Discrete Gaussian)**.** The Gaussian function with parameter $r > 0$ is $\rho_r(\mathbf{x}) = \exp(-\|\mathbf{x}\|^2/(2r^2))$. The discrete Gaussian distribution $D_{\mathbb{Z}^n,r}$ assigns to each $\mathbf{v} \in \mathbb{Z}^n$ the probability $\rho_r(\mathbf{v})/\rho_r(\mathbb{Z}^n)$. We write $D_{R,r}$ when sampling the coefficient vector of a ring element.

Lemma 2.4 (Tail Bounds [27])**.** *For $r > 0$ and positive integer n:* (i) $\Pr[\mathbf{v} \leftarrow D_{\mathbb{Z}^n,r} : \|\mathbf{v}\| > 2r\sqrt{n}] < 2^{-n}$*;* (ii) $\Pr[v \leftarrow D_{\mathbb{Z},r} : |v| > 12r] < 2^{-100}$.

2.4 Iterative Rejection Sampling

Iterative rejection sampling is the core technical tool that enables our scheme to achieve compact signatures. The central idea, due to Gärtner [14], is as follows: given a vector $\mathbf{y}$ sampled from some distribution and a short vector $\mathbf{v}$, a carefully designed pair of acceptance functions $f_{\mathbf{v}}$ and $g_{\mathbf{v}}$ governs whether to output $\mathbf{y} \pm \mathbf{v}$ or to reject ($\perp$), in such a way that the conditional output distribution exactly matches the input distribution.

Definition 2.5 (Rejection Procedure $R_{\mathbf{v}}$)**.** Given input $\mathbf{y}$, vector $\mathbf{v}$, and functions $f_{\mathbf{v}}, g_{\mathbf{v}}$: (1) Compute $(a, b) = (f_{\mathbf{v}}(\mathbf{y}), g_{\mathbf{v}}(\mathbf{y}))$. (2) Sample $\rho \xleftarrow{\$} [0, 1)$. (3) If $\rho < a$, return $\mathbf{y} - \mathbf{v}$. Else if $\rho < a + b$, return $\mathbf{y} + \mathbf{v}$. Else return $\perp$.

The functions $f_{\mathbf{v}}$ and $g_{\mathbf{v}}$ are constructed from the auxiliary function

$$S_{\mathbf{v}}(\mathbf{y}) = \sum_{k=0}^{\infty} (-1)^k \frac{\rho_r(\mathbf{y} + k\mathbf{v})}{\rho_r(\mathbf{y})}, \tag{1}$$

which depends only on $\langle \mathbf{y}, \mathbf{v} \rangle$ and $\|\mathbf{v}\|^2$ and can be approximated to precision $< 2^{-256}$ using only the first $\sim$20 terms [14].

Lemma 2.6 (Gärtner [14, Corollary 1])**.** *Let $\alpha = r/\|\mathbf{v}\|$ and define*

$$M_\alpha = 1 + \frac{2\alpha\sqrt{2\pi}\,\rho(\pi\alpha)}{\rho_\alpha(1) \cdot (1 - \rho(2\pi\alpha))}. \tag{2}$$

Then, for every $\mathbf{v}$ with $\alpha \leq r/\|\mathbf{v}\|$, the functions

$$f_{\mathbf{v}}(\mathbf{y}) = \begin{cases} S_{\mathbf{v}}(\mathbf{y})/M_\alpha & \text{if } \langle \mathbf{y}, \mathbf{v} \rangle \geq \|\mathbf{v}\|^2, \\ (1 - S_{\mathbf{v}}(-\mathbf{y}))/M_\alpha & \text{if } \langle \mathbf{y}, \mathbf{v} \rangle < \|\mathbf{v}\|^2, \end{cases} \tag{3}$$

$$g_{\mathbf{v}}(\mathbf{y}) = \begin{cases} (1 - S_{\mathbf{v}}(\mathbf{y}))/M_\alpha & \text{if } \langle \mathbf{y}, \mathbf{v} \rangle \geq -\|\mathbf{v}\|^2, \\ S_{\mathbf{v}}(-\mathbf{y})/M_\alpha & \text{if } \langle \mathbf{y}, \mathbf{v} \rangle < -\|\mathbf{v}\|^2, \end{cases} \tag{4}$$

fulfill the rejection sampling requirements: conditioned on $R_{\mathbf{v}}(\mathbf{y}) \neq \perp$, the output $\mathbf{z}$ follows the same distribution as $\mathbf{y}$, and the rejection probability is $1 - 1/M_\alpha$.

Remark 2.7 (Negligible Rejection)**.** When $\alpha \geq 4$, the rejection probability satisfies $1 - 1/M_\alpha < 2^{-108}$, and the rejection condition can be safely ignored.

2.5 Ring-Based Trapdoor Generation

The certificateless key structure requires the KGC to sample short preimages of identity-derived targets. The following ring-based trapdoor algorithms provide this capability.

Lemma 2.8 (Ring-TrapGen [29])**.** *Given a prime q and ring dimension n, there exists a PPT algorithm* $\mathsf{RingTrapGen}(q, n)$ *that outputs $a \in R_q$ together with a short basis T_a of $\Lambda_q^{\perp}(a) = \{(x_1, x_2) \in R^2 : x_1 + a \cdot x_2 \equiv 0 \pmod q\}$, satisfying $\|T_a\| \leq O(\sqrt{n \log q})$.*

Lemma 2.9 (Ring-SamplePre [29])**.** *Given $a \in R_q$ with trapdoor T_a, a Gaussian parameter $s_d \geq \left\|\widetilde{T}_a\right\| \cdot \omega(\sqrt{\log n})$, and a target $u \in R_q$, there exists a PPT algorithm* $\mathsf{RingSamplePre}(a, T_a, s_d, u)$ *that outputs $\mathbf{e} = (e_1, e_2) \in R^2$ satisfying $e_1 + a \cdot e_2 = u \bmod q$ and $\|\mathbf{e}\| \leq s_d\sqrt{2n}$.*

3 Scheme Definition and Security Model

This section first specifies the syntax of CL-NTRU-MS-IRS as a tuple of seven algorithms (Section 3.1), then formalizes the security requirements through two complementary adversary models—Type I and Type II—with epoch-scoped revocation oracles (Section 3.2).

3.1 Syntax of CL-NTRU-MS-IRS

Let $\mathcal{ID} = \{ID_1, \ldots, ID_N\}$ be the identity set of N signers. The certificateless multi-signature scheme CL-NTRU-MS-IRS consists of the following seven algorithms.

- $\mathsf{Setup}(1^n) \to (\mathsf{mpk}, \mathsf{msk})$. On input security parameter 1^n, the KGC generates a master public key mpk (containing $a \in R_q$ and public parameters) and a master secret key msk (the trapdoor T_a for a).
- $\mathsf{Partial\text{-}Key\text{-}Extract}(\mathsf{mpk}, \mathsf{msk}, ID_i) \to \mathbf{d}_i$. The KGC uses its trapdoor to generate a partial key $\mathbf{d}_i = (d_{i,1}, d_{i,2}) \in R^2$ for identity ID_i and delivers it to the user via a secure channel.
- $\mathsf{Set\text{-}User\text{-}Key}(\mathsf{mpk}, ID_i, \mathbf{d}_i) \to (\mathbf{x}_i, p_i)$. User i, upon receiving partial key $\mathbf{d}_i$ from the KGC, first verifies its correctness ($d_{i,1} + a \cdot d_{i,2} = H_0(ID_i)$ and $\|\mathbf{d}_i\| \leq B_d$), then independently generates a secret value $\mathbf{x}_i = (x_{i,1}, x_{i,2}) \in R^2$ and a corresponding user public value $p_i \in R_q$. The user's full secret key is $\mathsf{sk}_i = (\mathbf{d}_i, \mathbf{x}_i)$, and the full public key is $\mathsf{pk}_i = (ID_i, p_i)$.
- $\mathsf{MSign}(L, m, \mathsf{mpk})$. A two-round interactive protocol among N signers for message m, using the effective signing key $\tilde{\mathbf{s}}_i = \mathbf{d}_i + \mathbf{x}_i$.
- $\mathsf{AggSig}(\{(\mathbf{z}_i, \mathbf{b}_i)\}_{i=1}^N, c) \to \sigma$. Run by a designated combiner, aggregating partial signatures into a multi-signature $\sigma = (\mathbf{z}, c, \{\mathbf{b}_1, \ldots, \mathbf{b}_N\})$.
- $\mathsf{MVerify}(L, \sigma, m, \mathsf{mpk}) \to 0/1$. Verifies the multi-signature and outputs 1 (accept) or 0 (reject).

Remark 3.1 (Key Structure)**.** The effective signing key $\tilde{\mathbf{s}}_i = \mathbf{d}_i + \mathbf{x}_i$ derives its security from two independent sources: the KGC does not know $\mathbf{x}_i$ and therefore cannot compute $\tilde{\mathbf{s}}_i$ (defense against Type II adversaries); an external attacker does not know $\mathbf{d}_i$ (which requires the KGC's trapdoor to generate) and likewise cannot compute $\tilde{\mathbf{s}}_i$ (defense against Type I adversaries). The effective public key is $t_i = H_0(ID_i) + p_i \bmod q$, binding both identity information and the user's secret value.

3.2 Security Model

The security of a certificateless scheme must be established against two fundamentally different adversary classes [2]. A *Type I adversary* models an external attacker who does not hold the master secret but may corrupt users and replace their public keys. A *Type II adversary* models a malicious KGC that can generate partial keys for arbitrary identities but cannot access users' self-chosen secret values. We formalize each class below, incorporating epoch-scoped revocation oracles that model the ability to trigger key invalidation.

3.2.1 Type I Adversary (External Attacker)

A Type I adversary $\mathcal{A}_{\mathrm{I}}$ models an external attacker who does not possess the master secret but may corrupt users. Its capabilities include:

(a) *Partial key extraction*: Query the partial key $\mathbf{d}_i$ for any identity ID_i.

(b) *User key extraction*: Query the secret value $\mathbf{x}_i$ of any user i.

(c) *Public key replacement*: Replace user i's public value p_i with an arbitrary p_i' (without knowing the corresponding secret value). This models malicious key registration in the absence of a PKI.

(d) *Signing queries*: Request multi-signatures for any (L, m) involving an honest target signer.

(e) *Concurrent interaction*: Open multiple signing sessions simultaneously.

Definition 3.2 (Type I Security)**.** Consider the security game $\mathsf{Game}_{\mathrm{I}}^{\mathcal{A}_{\mathrm{I}}}(1^n)$:

1. Setup. The challenger runs $\mathsf{Setup}(1^n)$ and sends mpk to $\mathcal{A}_{\mathrm{I}}$, retaining msk. The initial epoch is $\mathsf{ep} = 0$ with $\mathsf{RA}_0 = \emptyset$.

2. Query phase. $\mathcal{A}_{\mathrm{I}}$ adaptively queries the following oracles:

- $\mathcal{O}_{\mathsf{PartialKey}}(ID_i)$: Returns partial key $\mathbf{d}_i$. Let $\mathcal{Q}_P$ be the set of extracted identities.
- $\mathcal{O}_{\mathsf{UserKey}}(ID_i)$: Returns user secret value $\mathbf{x}_i$. Let $\mathcal{Q}_U$ be the extraction set.
- $\mathcal{O}_{\mathsf{Replace}}(ID_i, p_i')$: Replaces user i's public value with p_i'.
- $\mathcal{O}_{\mathsf{Revoke}}(ID_i)$: Adds $H_{\mathsf{rev}}(ID_i)$ to the current revocation accumulator $\mathsf{RA}_{\mathsf{ep}}$. Let $\mathcal{Q}_R$ be the set of revoked identities.
- $\mathcal{O}_{\mathsf{Epoch}}()$: Advances the epoch counter $\mathsf{ep} \leftarrow \mathsf{ep} + 1$ and publishes the updated $\mathsf{RA}_{\mathsf{ep}}$.
- $\mathcal{O}_{\mathsf{Sign\text{-}1}}$, $\mathcal{O}_{\mathsf{Sign\text{-}2}}$: Two-round signing oracles (which internally check revocation status and bind to $(\mathsf{ep}, \mathsf{RA}_{\mathsf{ep}})$ via the challenge hash).
- $\mathcal{O}_{H_0}$, $\mathcal{O}_{H_1}$, $\mathcal{O}_{H_2}$, $\mathcal{O}_{H_{\mathsf{rev}}}$: Random oracles.

3. Forgery phase. $\mathcal{A}_{\mathrm{I}}$ outputs (m^*, L^*, σ^*), where σ^* includes an epoch ep^*.

$\mathcal{A}_{\mathrm{I}}$ wins if and only if: (i) $\mathsf{MVerify}(L^*, \sigma^*, m^*, \mathsf{mpk}) = 1$; (ii) L^* contains a target identity ID^* such that $ID^* \notin \mathcal{Q}_P \cup \mathcal{Q}_U$ (neither the partial key nor the user key of ID^* was extracted); (iii) ID^* was not revoked at epoch ep^* (i.e., $H_{\mathsf{rev}}(ID^*) \notin \mathsf{RA}_{\mathsf{ep}^*}$); (iv) (L^*, m^*) was not queried in any signing query at epoch ep^*.

The scheme has Type I EUF-CMA security if $\Pr[\mathcal{A}_{\mathrm{I}} \text{ wins}] \leq \mathsf{negl}(n)$ for every PPT adversary.

3.2.2 Type II Adversary (Malicious KGC)

A Type II adversary $\mathcal{A}_{\mathrm{II}}$ models a malicious KGC that can generate partial keys for arbitrary identities but does not know users' secret values. Its capabilities include:

(a) *Unrestricted partial key generation*: $\mathcal{A}_{\mathrm{II}}$ may query a partial key oracle $\mathcal{O}_{\mathsf{PartialKey}}(ID_i)$ for *any* identity with no restrictions (modeling the KGC's ability to compute partial keys at will). Let $\mathcal{Q}_P^{\mathrm{II}}$ be the set of queried identities.

(b) *User key extraction*: Query the secret value $\mathbf{x}_i$ of any user i.

(c) *Signing queries*: Request multi-signatures for any (L, m).

(d) *No public key replacement*: Unlike Type I, $\mathcal{A}_{\mathrm{II}}$ **cannot** replace public keys.

Remark 3.3 (Type II Power vs. Master Secret)**.** Rather than giving $\mathcal{A}_{\mathrm{II}}$ the raw master secret $\mathsf{msk} = T_a$ (which would prevent the reduction from embedding a Ring-SIS challenge as the master public key a), we grant $\mathcal{A}_{\mathrm{II}}$ *unrestricted oracle access* to partial key generation. This is operationally equivalent—any computation that $\mathcal{A}_{\mathrm{II}}$ could perform with T_a to derive partial keys is captured by the oracle—while enabling a clean security reduction. This formalization is standard in the certificateless literature [2].

Definition 3.4 (Type II Security)**.** Consider the security game $\mathsf{Game}_{\mathrm{II}}^{\mathcal{A}_{\mathrm{II}}}(1^n)$:

1. **Setup.** The challenger runs $\mathsf{Setup}(1^n)$ and sends mpk to $\mathcal{A}_{\mathrm{II}}$ (retaining msk). The initial epoch is $\mathsf{ep} = 0$.

2. **Query phase.** $\mathcal{A}_{\mathrm{II}}$ adaptively queries:

   - $\mathcal{O}_{\mathsf{PartialKey}}(ID_i)$: Returns partial key $\mathbf{d}_i$ for *any* identity, with no restrictions.
   - $\mathcal{O}_{\mathsf{UserKey}}(ID_i)$: Returns $\mathbf{x}_i$. Let $\mathcal{Q}_U$ be the extraction set.
   - $\mathcal{O}_{\mathsf{Revoke}}(ID_i)$, $\mathcal{O}_{\mathsf{Epoch}}()$: As in the Type I game.
   - $\mathcal{O}_{\mathsf{Sign\text{-}1}}$, $\mathcal{O}_{\mathsf{Sign\text{-}2}}$: Two-round signing oracles.
   - Random oracles.

3. **Forgery phase.** $\mathcal{A}_{\mathrm{II}}$ outputs (m^*, L^*, σ^*) with epoch ep^*.

$\mathcal{A}_{\mathrm{II}}$ wins if and only if: (i) $\mathsf{MVerify}(L^*, \sigma^*, m^*, \mathsf{mpk}) = 1$; (ii) L^* contains a target identity ID^* such that $ID^* \notin \mathcal{Q}_U$; (iii) ID^* was not revoked at epoch ep^*; (iv) (L^*, m^*) was not queried in any signing query at epoch ep^*.

The scheme has Type II EUF-CMA security if $\Pr[\mathcal{A}_{\mathrm{II}} \text{ wins}] \leq \mathsf{negl}(n)$ for every PPT adversary.

Remark 3.5 (Revocation and Security)**.** The revocation oracle $\mathcal{O}_{\mathsf{Revoke}}$ allows the adversary to revoke arbitrary identities, modeling a realistic scenario in which an attacker can trigger revocations (e.g., by compromising a device). The winning condition (iii) ensures that the target signer must be *unrevoked* at the forgery epoch—forging a signature for an already-revoked signer is not a meaningful attack. The challenge hash binding $c = H_2(w, m, L, \mathsf{ep}, \mathsf{RA}_{\mathsf{ep}})$ ensures that a signature valid at one epoch cannot be "migrated" to a different epoch with a different revocation state: changing ep or $\mathsf{RA}_{\mathsf{ep}}$ changes c, which breaks the verification equation.

Remark 3.6 (Complementarity of the Two Adversary Types)**.** Type I and Type II adversaries have complementary capabilities: Type I does not know the master secret but can replace public keys (external attacker perspective); Type II knows the master secret but cannot replace public keys (internal malicious entity perspective). The scheme must resist both types simultaneously to provide complete security guarantees in the certificateless framework.

4 Construction

This section presents CL-NTRU-MS-IRS in full detail. We describe the seven constituent algorithms in logical order: system setup (Section 4.1), partial key extraction (Section 4.2), user key setup (Section 4.3), the two-round signing protocol with epoch-scoped revocation checks (Section 4.4), the iterative rejection sampling subroutine (Section 4.5), signature aggregation (Section 4.6), verification (Section 4.7), and a design rationale for the aggregation strategy (Section 4.8).

4.1 System Setup

$\mathsf{Setup}(1^n)$: The KGC, given security parameter 1^n, proceeds as follows.

1. Select parameters: prime q, ring dimension n (power of 2), partial key sampling parameter s_d, user key sampling parameter σ_u, signature Gaussian parameter r, challenge Hamming weight κ, and epoch length ΔT.
2. Run $\mathsf{RingTrapGen}(q, n) \to (a, T_a)$, obtaining master public key $a \in R_q$ and trapdoor T_a. Set master secret key $\mathsf{msk} = T_a$.
3. Define the challenge space $\mathcal{C} = \{c \in R : \text{coefficients in } \{0,1\},\ \|c\|_1 = \kappa\}$.
4. Select hash functions with explicit domain separation tags:
   - $H_0 : \{0,1\}^* \to R_q$, defined as $H_0(x) = \mathcal{H}(\texttt{0x00}\|x)$ (identity hash),
   - $H_1 : \{0,1\}^* \to \{0,1\}^{2\lambda}$, defined as $H_1(x) = \mathcal{H}(\texttt{0x01}\|x)$ (commitment hash, λ: security parameter),
   - $H_2 : \{0,1\}^* \to \mathcal{C}$, defined as $H_2(x) = \mathcal{H}'(\texttt{0x02}\|x)$ (challenge hash),
   - $H_{\mathsf{rev}} : \{0,1\}^* \to \{0,1\}^{\lambda}$, defined as $H_{\mathsf{rev}}(x) = \mathcal{H}''(\texttt{0x03}\|x)$ (revocation tag hash),

   where $\mathcal{H}$, $\mathcal{H}'$, $\mathcal{H}''$ denote a single underlying hash function (e.g., SHAKE-256) with appropriate output processing. The one-byte domain tags ensure that the four hash functions behave as independent random oracles in the security proof, even when instantiated from a common primitive.
5. Compute bounds: $B_d = s_d\sqrt{2n}$ (partial key norm bound), $B_x = \sigma_u\sqrt{2n+4}$ (user key norm bound), $B_k = B_d + B_x$ (effective key norm bound), and $B_s = 2Nr\sqrt{2n}$ (aggregated signature norm bound; see Proposition 5.3).
6. Compute $\alpha = r/B_k$ and $M = M_\alpha$ via (2).
7. Initialize the revocation state: set epoch counter $\mathsf{ep} \leftarrow 0$ and revocation accumulator $\mathsf{RA}_0 \leftarrow \emptyset$.

Output $\mathsf{mpk} = (a, n, q, H_0, H_1, H_2, H_{\mathsf{rev}}, r, s_d, \sigma_u, \kappa, B_k, B_s, \alpha, \Delta T)$ and $\mathsf{msk} = T_a$.

Remark 4.1 (Epoch-Based Revocation Without Key Update)**.** At the start of each epoch ep, the KGC publishes an updated revocation accumulator $\mathsf{RA}_{\mathsf{ep}}$ containing the revocation tags $H_{\mathsf{rev}}(ID)$ of all revoked identities. This accumulator can be instantiated as a Merkle hash tree over the revoked tag set, enabling compact (non-)membership proofs. Importantly, *no key update is performed*: unrevoked signers continue to use their original keys $(\mathbf{d}_i, \mathbf{x}_i)$ across all epochs. Revocation takes effect solely through the challenge hash binding—H_2 includes both ep and $\mathsf{RA}_{\mathsf{ep}}$ as inputs (see Section 4.4), so a signature generated in one epoch is cryptographically tied to that epoch's revocation state and cannot be verified under a different state. This design avoids the overhead and complexity of periodic key renewal while still providing timely revocation guarantees.

4.2 Partial Key Extraction

$\mathsf{Partial\text{-}Key\text{-}Extract}(\mathsf{mpk}, \mathsf{msk}, ID_i)$: The KGC generates a partial key for identity ID_i.

1. Compute the identity hash $\tau_i = H_0(ID_i) \in R_q$.
2. Run $\mathsf{RingSamplePre}(a, T_a, s_d, \tau_i) \to \mathbf{d}_i = (d_{i,1}, d_{i,2})$, such that $d_{i,1} + a \cdot d_{i,2} = \tau_i \bmod q$ and $\|\mathbf{d}_i\| \le B_d$.
3. Deliver $\mathbf{d}_i$ to user i via a secure channel.

4.3 User Key Setup

$\mathsf{Set\text{-}User\text{-}Key}(\mathsf{mpk}, ID_i, \mathbf{d}_i)$: User i, upon receiving the partial key $\mathbf{d}_i$ from the KGC, proceeds as follows.

1. *Partial key verification.* Compute $\tau_i = H_0(ID_i)$ and verify that $d_{i,1} + a \cdot d_{i,2} = \tau_i \bmod q$ and $\|\mathbf{d}_i\| \le B_d$. If either check fails, reject the partial key. This prevents a malicious or faulty KGC from issuing invalid partial keys.
2. Sample $\mathbf{x}_i = (x_{i,1}, x_{i,2}) \leftarrow D^2_{R,\sigma_u}$.
3. If $\|\mathbf{x}_i\| > B_x$, go to step 2.
4. Compute user public value $p_i = x_{i,1} + a \cdot x_{i,2} \bmod q$.
5. Set the effective signing key $\tilde{\mathbf{s}}_i = \mathbf{d}_i + \mathbf{x}_i = (d_{i,1} + x_{i,1},\ d_{i,2} + x_{i,2})$.

Output full public key $\mathsf{pk}_i = (ID_i, p_i)$ and full secret key $\mathsf{sk}_i = (\mathbf{d}_i, \mathbf{x}_i)$.

Remark 4.2 (Effective Key Norm)**.** By the triangle inequality, $\|\tilde{\mathbf{s}}_i\| = \|\mathbf{d}_i + \mathbf{x}_i\| \le \|\mathbf{d}_i\| + \|\mathbf{x}_i\| \le B_d + B_x = B_k$. This bound holds unconditionally given that steps 1 and 3 pass, so no additional rejection step is needed for the effective key.

Remark 4.3 (Effective Public Key)**.** Define the effective public key $t_i = \tau_i + p_i = H_0(ID_i) + p_i \bmod q$. By construction:

$$\tilde{s}_{i,1} + a \cdot \tilde{s}_{i,2} = (d_{i,1} + x_{i,1}) + a \cdot (d_{i,2} + x_{i,2}) = \tau_i + p_i = t_i \pmod q.$$

From the verification perspective, t_i plays the same role as the public key in the original scheme, but its construction fuses identity information (τ_i) with a user-chosen component (p_i), combining identity binding with key autonomy.

4.4 Two-Round Multi-Signature Protocol

Let $L = \{(ID_1, p_1), \ldots, (ID_N, p_N)\}$ be the signer public key set, m the message, and ep the current epoch with revocation accumulator $\mathsf{RA}_{\mathsf{ep}}$ (published by the KGC and available to all participants). Each signer i holds the effective signing key $\tilde{\mathbf{s}}_i = \mathbf{d}_i + \mathbf{x}_i$.

Convention. The signer set L is always encoded in a canonical (deterministic) order—e.g., lexicographic by identity ID_i—before being input to any hash function. This ensures that all signers and the verifier compute identical hash values. We write $\mathsf{Canon}(L)$ for this canonical encoding, though we omit the notation in hash arguments for readability.

4.4.1 Round 1: Commitment

Each signer i executes:

(1) *Revocation check.* Compute the revocation tag $u_i = H_{\mathsf{rev}}(ID_i)$. Verify that $u_i \notin \mathsf{RA}_{\mathsf{ep}}$ (i.e., the signer has not been revoked in the current epoch). If revoked, abort.

(2) Sample $\mathbf{y}_i = (y_{i,1}, y_{i,2}) \leftarrow D^2_{R,r}$.

(3) Compute commitment $w_i = y_{i,1} + a \cdot y_{i,2} \bmod q$.

(4) Compute commitment hash $\delta_i = H_1(w_i \parallel ID_i \parallel L \parallel \mathsf{ep}) \in \{0,1\}^{2\lambda}$.

(5) **Broadcast** δ_i to all other signers (via the combiner or direct peer-to-peer).

4.4.2 Round 2: Response

Upon receiving all $\{\delta_j\}_{j \neq i}$, signer i executes:

(1) **Broadcast** w_i to all other signers.

(2) For each j, verify $\delta_j = H_1(w_j \parallel ID_j \parallel L \parallel \mathsf{ep})$. If any check fails, **abort**.

(3) Compute aggregate commitment $w = \sum_{j=1}^{N} w_j \bmod q$.

(4) Compute challenge $c = H_2(w,\ m,\ L,\ \mathsf{ep},\ \mathsf{RA}_{\mathsf{ep}}) \in \mathcal{C}$.

(5) Run $\texttt{IterRejectSample}(\mathbf{y}_i, \tilde{\mathbf{s}}_i, c) \to (\mathbf{z}_i, \mathbf{b}_i)$ or $\perp$.

(6) If $\perp$: **broadcast a** RESTART **signal** to all other signers, **discard** the current $\mathbf{y}_i$, and return to Round 1 with fresh randomness. Upon receiving a RESTART from any signer, all participants must likewise discard their current state and restart. This mandatory full-group restart is necessary because the challenge c depends on the aggregate commitment w, which in turn depends on every signer's w_i; see Remark 5.6 for a quantitative analysis.

(7) Send $(\mathbf{z}_i, \mathbf{b}_i)$ to the designated combiner.

Remark 4.4 (Challenge Binding)**.** The challenge hash $c = H_2(w, m, L, \mathsf{ep}, \mathsf{RA}_{\mathsf{ep}})$ binds the signature to five independent components: the aggregate commitment w (ensuring commitment integrity), the message m (standard unforgeability), the signer set L (preventing rogue key manipulation), the epoch ep (temporal freshness), and the revocation accumulator $\mathsf{RA}_{\mathsf{ep}}$ (ensuring that the signature is valid only under the current revocation state). A signature generated in epoch ep cannot be verified against a different epoch's revocation state, effectively invalidating signatures involving subsequently revoked signers.

4.5 Iterative Rejection Sampling

4.6 Signature Aggregation

AggSig: The designated combiner receives all $(\mathbf{z}_i, \mathbf{b}_i)$, computes $\mathbf{z} = \sum_{i=1}^{N} \mathbf{z}_i$, checks $\|\mathbf{z}\| \leq B_s$, and outputs $\sigma = (\mathbf{z},\ c,\ \{\mathbf{b}_1, \ldots, \mathbf{b}_N\},\ \mathsf{ep})$.

4.7 Verification

$\mathsf{MVerify}(L, \sigma, m, \mathsf{mpk})$: Given $L = \{(ID_1, p_1), \ldots, (ID_N, p_N)\}$, multi-signature $\sigma = (\mathbf{z}, c, \{\mathbf{b}_i\}_{i=1}^{N}, \mathsf{ep})$, and message m:

(1) *Epoch and revocation check.* Verify that epoch ep is current or within an acceptable validity window. Retrieve the revocation accumulator $\mathsf{RA}_{\mathsf{ep}}$ from the KGC's public bulletin board (or any mirror thereof). For each $i \in [N]$, verify that $H_{\mathsf{rev}}(ID_i) \notin \mathsf{RA}_{\mathsf{ep}}$. If any signer is revoked or ep is invalid, output 0.

(2) *Norm check.* If $\|\mathbf{z}\| > B_s$, output 0.

(3) For each $i \in [N]$, compute effective public key $t_i = H_0(ID_i) + p_i \bmod q$.

(4) For each $i \in [N]$, reconstruct the ternary polynomial $c'_i \in R$ from $(c, \mathbf{b}_i)$: for each position j with $c_j = 1$, set $c'_{i,j} = (-1)^{1-b_{i,j}}$; for positions with $c_j = 0$, set $c'_{i,j} = 0$.

Algorithm 1 `IterRejectSample`$(\mathbf{y}, \tilde{\mathbf{s}}, c)$

Input: Random vector $\mathbf{y} \in R^2$, effective signing key $\tilde{\mathbf{s}} = (\tilde{s}_1, \tilde{s}_2) \in R^2$, challenge $c \in \mathcal{C}$
Output: $(\mathbf{z}, \mathbf{b}) \in R^2 \times \{0,1\}^\kappa$ or $\perp$

```
J ← {j : c_j = 1}                                  ▷ Non-zero coefficient positions; |J| ≤ κ
z_0 ← y, k ← 0
for each j ∈ J in increasing order do
    v ← x^j · s̃ = (x^j · s̃_1, x^j · s̃_2)                          ▷ ‖v‖ = ‖s̃‖ in R
    k ← k + 1
    z_k ← R_v(z_{k−1})                              ▷ Rejection sampling step (Definition 2.5)
    if z_k = ⊥ then return ⊥
    end if
    if z_k = z_{k−1} − v then b_j ← 0
    else b_j ← 1
    end if
end for
z ← z_k, b ← (b_j)_{j∈J}
Construct c′ ∈ R: for j ∈ J, set c′_j = (−1)^{1−b_j}; for j ∉ J, set c′_j = 0
return (z, b)                                      ▷ z = y + s̃ · c′ with c′ ≡ c (mod 2)
```

(5) Compute $T = \sum_{i=1}^{N} t_i \cdot c'_i \bmod q$.

(6) Compute $w' = z^{(1)} + a \cdot z^{(2)} - T \bmod q$.

(7) If $c = H_2(w',\ m,\ L,\ \mathsf{ep},\ \mathsf{RA}_{\mathsf{ep}})$, output 1; otherwise output 0.

4.8 Discussion: Aggregation Strategy and Design Rationale

The choice of aggregation strategy deserves explicit justification, as it critically affects signature compactness.

Our approach: full vector aggregation with IRS sign-bit output. In our construction, the master public key element $a \in R_q$ is a *global* system parameter shared by all signers. The verification equation

$$z^{(1)} + a \cdot z^{(2)} = w + \sum_{i=1}^{N} t_i \cdot c'_i \pmod q$$

has the property that a multiplies the *aggregated* second component $z^{(2)} = \sum_i z_{i,2}$, not individual per-signer values. This allows *both* components of $\mathbf{z}$ to be fully summed into a single two-dimensional vector. The per-signer sign-bit vectors $\mathbf{b}_i$ are a natural output of the iterative rejection sampling process: at each step, IRS evaluates the acceptance functions $f_{\mathbf{v}}$ and $g_{\mathbf{v}}$ to *jointly* determine whether to add or subtract the rotated key vector and whether to accept or reject, favoring the direction that yields a shorter output $\mathbf{z}_i$. The sign bits faithfully record these intelligent decisions, enabling the verifier to reconstruct the exact ternary polynomial c'_i used by each signer. In other words, the sign bits are not an overhead imposed by the aggregation strategy; they are an intrinsic record of IRS's signature optimization, which must be communicated to the verifier in any setting where c and c'_i are not algebraically equivalent from the verifier's perspective.

Alternative: per-signer matrix partial aggregation (mod $2q$). An alternative design, natural in the NTRU/BLISS framework, uses per-signer matrices $A_i = [2pk_i,\ (q-2)]$ with verification modulo $2q$. In this setting, the key relation $A_i\mathbf{s}_i \equiv q \pmod{2q}$ ensures that $q \cdot c'_i \equiv q \cdot c$

(mod $2q$) (since $c_i' - c = 2k$ and $2qk \equiv 0 \pmod{2q}$), so the verifier does *not* need sign bits—the modular arithmetic automatically absorbs the difference between c and c_i'. However, expanding $\sum_i A_i \mathbf{z}_i$ reveals that the second component's coefficient $(q-2)$ is a global constant and can be factored out of the sum, while the first component's coefficient pk_i is per-signer and prevents aggregation. This yields $N+1$ ring elements: N individual first-components $(z_{1,1}, \ldots, z_{N,1})$ plus one aggregated second-component $z_{\mathsf{agg},2} = \sum_i z_{i,2}$.

The mod-q vs. mod-$2q$ trade-off. The two approaches reflect a fundamental design trade-off. Working modulo $2q$ eliminates sign bits (because $2qk \equiv 0 \pmod{2q}$) but forces per-signer public key matrices that block full vector aggregation. Working modulo q requires sign bits (because $2t_i k_i \not\equiv 0 \pmod{q}$ for odd prime q) but permits a global a that enables full vector aggregation. In both settings, IRS performs the *same* intelligent sign selection—the difference lies solely in whether the verifier can infer the sign choices algebraically or must receive them explicitly.

Quantitative comparison. For Parameter Set I ($n = 512$, $q = 12289$) with $N = 10$ signers:

Strategy	**Signature size**	**Ratio**
Full vector agg. + IRS sign bits (ours)	$\approx$ 914 bytes	1.0×
Per-signer matrix partial agg.	$\approx$ 4,250 bytes	4.6×
No aggregation (all $2N$ components)	$\approx$ 7,750 bytes	8.5×

Our approach achieves a 4.6× compactness advantage over the per-signer-matrix alternative. The key enabler is the *global* structure of the master public key a: because a is shared across all signers, the bilinear structure of $z^{(1)} + a \cdot z^{(2)}$ permits full summation of both components. The per-signer sign bits ($N\kappa = 580$ bits ≈ 73 bytes for $N = 10$, $\kappa = 58$) are lightweight: they record IRS's per-coefficient sign decisions that simultaneously optimize signature compactness and maintain distributional independence from the secret key. The net result is a 4.6× size reduction at the expense of 73 bytes of IRS metadata—a favorable trade-off by any measure.

Remark 4.5 (IRS Sign Selection and Aggregation Compatibility)**.** Full vector aggregation is particularly well-suited to the iterative rejection sampling framework. The sign bits are not an artifact of the aggregation strategy but an intrinsic output of IRS: at each iterative step, the acceptance functions $f_{\mathbf{v}}$ and $g_{\mathbf{v}}$ *jointly* decide the sign direction and acceptance, favoring the choice that yields a shorter $\mathbf{z}_i$. This is the fundamental mechanism by which IRS surpasses classical approaches such as BLISS [5], which first selects a sign uniformly at random and then decides whether to accept—decoupling two decisions that IRS treats as one. Since each $\mathbf{z}_i$ follows $D_{R^2,r}$ independently of $\tilde{\mathbf{s}}_i$ (Lemma 6.8), the aggregated $\mathbf{z} = \sum_i \mathbf{z}_i$ inherits a clean distributional characterization (approximately $D_{R^2,r\sqrt{N}}$), which simplifies both the correctness analysis (Proposition 5.3) and the signature simulation in the security proof (Game G_2).

5 Correctness

Before turning to security, we establish that the scheme is correct: honestly generated multi-signatures always pass verification, the aggregated signature satisfies the required norm bound with overwhelming probability, and the signing process terminates efficiently. The argument is decomposed into three self-contained propositions, which together prove the following theorem.

Theorem 5.1 (Correctness)**.** *Let the system parameters satisfy $B_s \geq 2Nr\sqrt{2n}$. If all N signers honestly execute the protocol and no iterative rejection sampling instance outputs $\perp$, then the resulting multi-signature passes verification with probability* 1*. The signing process (with restarts upon rejection) terminates in expected M_α^κ attempts per signer.*

The proof is split into three sub-propositions.

5.1 Algebraic Correctness of Verification

Proposition 5.2 (Algebraic Correctness)**.** *If the iterative rejection sampling successfully outputs* $(\mathbf{z}_i, \mathbf{b}_i)$ *for every signer* i *(i.e.,* $\mathbf{z}_i \neq \perp$*), then the value* w' *reconstructed during verification equals the aggregate commitment* w*.*

Proof. By the construction in Algorithm 1, signer i's output satisfies

$$\mathbf{z}_i = \mathbf{y}_i + \tilde{\mathbf{s}}_i \cdot c'_i, \tag{5}$$

where c'_i is a ternary polynomial determined by the sign bits $\mathbf{b}_i$ and the challenge c, satisfying $c'_i \equiv c \pmod 2$. The coefficients of c'_i at the non-zero positions of c are ± 1, with the sign at each position chosen by IRS's acceptance functions $f_{\mathbf{v}}$ and $g_{\mathbf{v}}$ to favor shorter output vectors while preserving distributional independence from the secret key.

A subtle but important point concerns how the verifier handles the per-signer variation in c'_i. The iterative rejection sampling produces a *different* ternary polynomial c'_i for each signer—this is not a defect but a feature: the sign choices at each step are the mechanism by which IRS achieves low rejection rates and compact signatures. Writing $c'_i = c + 2k_i$ for some $k_i \in R$, we observe that in a mod-$2q$ framework (as in Gärtner's single-signer scheme [14]), the term $2k_i$ would be absorbed by the modular arithmetic ($2qk_i \equiv 0 \pmod{2q}$), making sign bits unnecessary. In our mod-q framework, however, $2t_ik_i \not\equiv 0 \pmod q$ for odd prime q, so the verifier cannot infer c'_i from c alone. The sign-bit vectors $\mathbf{b}_i$ therefore serve as explicit records of IRS's sign decisions, enabling the verifier to reconstruct each c'_i exactly and complete the verification.

Writing $\mathbf{z}_i = (z_{i,1}, z_{i,2})$, $\mathbf{y}_i = (y_{i,1}, y_{i,2})$, $\tilde{\mathbf{s}}_i = (\tilde{s}_{i,1}, \tilde{s}_{i,2})$:

$$\begin{aligned} z_{i,1} + a \cdot z_{i,2} &= (y_{i,1} + \tilde{s}_{i,1} \cdot c'_i) + a \cdot (y_{i,2} + \tilde{s}_{i,2} \cdot c'_i) \\ &= \underbrace{(y_{i,1} + a \cdot y_{i,2})}_{=w_i} + \underbrace{(\tilde{s}_{i,1} + a \cdot \tilde{s}_{i,2})}_{=t_i} \cdot c'_i \pmod q. \end{aligned} \tag{6}$$

The second equality uses the key algebraic relation $\tilde{s}_{i,1} + a \cdot \tilde{s}_{i,2} = t_i \pmod q$, which holds by the construction of the dual-source key (Section 4.3).

For the aggregated signature $\mathbf{z} = \sum_{i=1}^{N} \mathbf{z}_i$, writing $\mathbf{z} = (z^{(1)}, z^{(2)})$:

$$\begin{aligned} z^{(1)} + a \cdot z^{(2)} &= \sum_{i=1}^{N} \big(z_{i,1} + a \cdot z_{i,2}\big) \\ &= \sum_{i=1}^{N} (w_i + t_i \cdot c'_i) = \underbrace{\sum_{i=1}^{N} w_i}_{=w} + \underbrace{\sum_{i=1}^{N} t_i \cdot c'_i}_{=T} \pmod q. \end{aligned} \tag{7}$$

Note that the linearity of both the ring operations and the modular reduction is essential here: the global element a factors out of the sum $\sum_i z_{i,2}$, allowing the two components of $\mathbf{z}$ to be aggregated independently.

The verifier reconstructs each c'_i from $(c, \mathbf{b}_i)$, computes $T = \sum_i t_i \cdot c'_i$, and obtains

$$w' = z^{(1)} + a \cdot z^{(2)} - T = w + T - T = w \pmod q.$$

Since the challenge hash binds to the epoch and revocation state, verification confirms

$$H_2(w',\ m,\ L,\ \mathsf{ep},\ \mathsf{RA}_{\mathsf{ep}}) = H_2(w,\ m,\ L,\ \mathsf{ep},\ \mathsf{RA}_{\mathsf{ep}}) = c,$$

and the check passes. □

5.2 Probabilistic Bound on Signature Norm

Proposition 5.3 (Norm Constraint)**.** *Let $B_s = 2Nr\sqrt{2n}$. If all iterative rejection sampling steps succeed ($\mathbf{z}_i \neq \perp$ for all i), then* $\Pr[\|\mathbf{z}\| > B_s] \leq N \cdot 2^{-2n}$.

Proof. By Lemma 2.6, conditioned on $R_{\mathbf{v}}(\mathbf{z}_{k-1}) \neq \perp$, the output $\mathbf{z}_k$ follows the same distribution as $\mathbf{z}_{k-1}$. By induction, if $\mathbf{z}_0 = \mathbf{y}_i \sim D_{R^2,r}$, then the final output $\mathbf{z}_i$ (conditioned on all steps not rejecting) also follows $D_{R^2,r}$.

More precisely, the coefficient vector of $\mathbf{z}_i$ is a $2n$-dimensional vector following $D_{\mathbb{Z}^{2n},r}$. By Lemma 2.4(i):

$$\Pr[\|\mathbf{z}_i\| > 2r\sqrt{2n}] < 2^{-2n}. \tag{8}$$

By the triangle inequality, $\|\mathbf{z}\| \leq \sum_{i=1}^{N} \|\mathbf{z}_i\|$. Defining $E_i = \{\|\mathbf{z}_i\| \leq 2r\sqrt{2n}\}$ and applying the union bound: $\Pr[\bigcap_{i=1}^{N} E_i] \geq 1 - N \cdot 2^{-2n}$. Under $\bigcap_{i=1}^{N} E_i$, we have $\|\mathbf{z}\| \leq 2Nr\sqrt{2n} = B_s$. For Parameter Set I ($n = 512$), $2^{-2n} = 2^{-1024}$, so this probability is negligible even for large N.

Remark 5.4 (Tighter Norm Bound)**.** The above uses a term-by-term triangle inequality bound. Exploiting the independence of the $\mathbf{z}_i$ and the additivity of Gaussian distributions, one can show that $\mathbf{z} = \sum_i \mathbf{z}_i$ approximately follows $D_{\mathbb{Z}^{2n},r\sqrt{N}}$, so $\|\mathbf{z}\| \leq 2r\sqrt{2nN}$ with overwhelming probability. This permits the smaller bound $B_s = 2r\sqrt{2nN}$ (vs. $2Nr\sqrt{2n}$ from the triangle inequality). For simplicity, the main theorem uses the triangle inequality bound.

□

5.3 Termination and Expected Restart Count

Proposition 5.5 (Termination)**.** *For a single signer, `IterRejectSample` succeeds with probability $p_{\text{succ}} = M_\alpha^{-\kappa}$. The signing process (with restarts) terminates in expected M_α^{κ} attempts. For Parameter Set I ($\alpha \approx 1.54$, $\kappa = 58$), $p_{\text{succ}} \approx 0.53$ and the expected number of restarts is about* 0.88.

Proof. Algorithm 1 has $\kappa_c \leq \kappa$ iterative steps. Each step $R_{\mathbf{v}}$ succeeds with probability $1/M_\alpha$ (Lemma 2.6). Since, conditioned on non-rejection, $\mathbf{z}_k \sim D_{R^2,r}$ (independent of $\mathbf{v}$), the rejection probability at each step is constantly $1 - 1/M_\alpha$. Hence all κ_c steps succeed with probability $p_{\text{succ}} = M_\alpha^{-\kappa_c}$.

The signing algorithm restarts from round 1 upon failure. Each restart is independent, so the number of attempts follows a geometric distribution with mean M_α^{κ}.

For Parameter Set I: $\alpha = 128/83.3 \approx 1.536$. Computing via (2): $M_\alpha \approx 1.0109$, thus $p_{\text{succ}} \approx 1.0109^{-58} \approx 0.531$, and the expected number of restarts is ≈ 0.88. □

Remark 5.6 (Restart Semantics)**.** If signer i's iterative rejection sampling outputs $\perp$, a new signing attempt is required. Because the challenge $c = H_2(w, m, L, \mathsf{ep}, \mathsf{RA}_{\mathsf{ep}})$ depends on the aggregate commitment $w = \sum_j w_j$, signer i cannot simply re-sample $\mathbf{y}_i$ while keeping others' commitments fixed—doing so would change w_i and hence w, invalidating the challenge. Consequently, *all signers must discard their current state and restart from Round 1 with fresh randomness.*

This is inherent to any two-round Fiat-Shamir multi-signature scheme in which the challenge depends on all commitments (including DOTT [11] and MuSig-L [22]). The practical impact can be mitigated through two strategies: (i) *pipelining*, in which each signer pre-computes and stores multiple commitment tuples $(\mathbf{y}_i^{(k)}, w_i^{(k)}, \delta_i^{(k)})$ so that a restart merely advances to the next pre-computed tuple without fresh Gaussian sampling; and (ii) selecting parameters with low overall rejection probability (e.g., Parameter Set I achieves $p_{\text{succ}} \approx 0.53$ per signer, so with $N = 10$ signers the probability that *all* succeed in a single attempt is $\approx 0.53^{10} \approx 0.002$, requiring on average ≈ 600 group attempts—motivating Parameter Set III where $p_{\text{succ}} \approx 1$ and restarts are eliminated entirely).

Combining Propositions 5.2–5.5 completes the proof of Theorem 5.1. □

6 Security Analysis

This section presents the main theoretical results. We begin by stating the two security theorems and comparing their structures (Section 6.1). Section 6.2 gives the full Type I game-sequence proof. Section 6.3 gives the Type II proof, preceded by a distribution lemma that justifies the trapdoor-free simulation. Section 6.4 establishes the key-independent distribution and sign-bit simulatability properties of the iterative rejection sampling. Section 6.5 discusses reduction tightness.

6.1 Main Theorems

Theorem 6.1 (Type I EUF-CMA Security)**.** *Let H_0, H_1, H_2, H_{rev} be modeled as independent random oracles (via domain separation). If there exists a PPT Type I adversary $\mathcal{A}_{\mathrm{I}}$ with non-negligible advantage ϵ_{I} in the game of Definition 3.2,* restricted to forgeries in which no signer in L^* has a replaced public key with unknown effective secret key *(i.e., $\mathcal{R} = \emptyset$ in the notation of the proof), then there exists an algorithm $\mathcal{B}_{\mathrm{I}}$ that solves the* $\mathrm{RSIS}_{q,\beta}$ *problem (with $\beta \leq 2B_s + 2\kappa B_k(|L^*| + 1)$) with probability at least*

$$\Pr[\mathcal{B}_{\mathrm{I}} \textit{ succeeds}] \geq \frac{1}{q_P + 1} \cdot \left(\frac{\epsilon_{\mathrm{I}}^2}{q_H + q_S} - \frac{1}{|\mathcal{C}|} \right) - 2^{-\kappa},$$

where q_P is the number of partial key extraction queries, q_H the total number of hash queries, and q_S the number of signing queries. The case $\mathcal{R} \neq \emptyset$ (replaced keys in the forgery set) remains an open problem; see Remark 6.5.

Theorem 6.2 (Type II EUF-CMA Security)**.** *Under the same random oracle modeling, if there exists a PPT Type II adversary $\mathcal{A}_{\mathrm{II}}$ with non-negligible advantage ϵ_{II} in the game of Definition 3.4, then there exists an algorithm $\mathcal{B}_{\mathrm{II}}$ that solves the* $\mathrm{RSIS}_{q,\beta'}$ *problem (with $\beta' \leq 2B_s + 2\kappa B_k(|L^*| + 1)$) with probability at least*

$$\Pr[\mathcal{B}_{\mathrm{II}} \textit{ succeeds}] \geq \frac{1}{q_U + 1} \cdot \left(\frac{\epsilon_{\mathrm{II}}^2}{q_H + q_S} - \frac{1}{|\mathcal{C}|} \right) - 2^{-\kappa} - 2^{-\Omega(n)},$$

where q_U is the number of user key extraction queries and $2^{-\Omega(n)}$ accounts for the statistical distance introduced by the H_0-programming simulation (Lemma 6.6). This reduction is complete (no public key replacement gap).

Remark 6.3 (Comparison of the Two Reductions)**.** Both proofs share a common backbone: game sequence, signature simulation via IRS distribution independence, forking extraction, and Ring-SIS solution. The key differences are as follows. For Type I, $\mathcal{B}_{\mathrm{I}}$ retains the real trapdoor T_a for partial key generation and embeds the Ring-SIS challenge as a; the target guess is over partial key queries (q_P); however, the reduction is *incomplete* when the forgery set L^* includes signers with replaced public keys (Remark 6.5). For Type II, $\mathcal{B}_{\mathrm{II}}$ also embeds the Ring-SIS challenge as a but answers partial key queries by programming H_0 (Lemma 6.6); the target guess is over user key queries (q_U); since $\mathcal{A}_{\mathrm{II}}$ cannot replace keys, the extraction is complete.

6.2 Proof of Theorem 6.1 (Type I)

Proof. The proof proceeds via a sequence of games $\mathsf{G}_0, \ldots, \mathsf{G}_4$, progressively transforming the real security game into a form from which a Ring-SIS solution can be extracted. We write W_i for the event that $\mathcal{A}_{\mathrm{I}}$ wins in G_i.

Game G_0 (Real game). This is the original security game from Definition 3.2 (for the Type I proof; the Type II proof follows analogously from Definition 3.4). $\Pr[W_0] = \epsilon$.

Game G_1 (Guess the target signer). $\mathcal{C}$ uniformly samples a target index $\mathsf{idx}^* \xleftarrow{\$} [q_P + 1]$ at the start. It maintains a counter $\mathsf{ctr} = 0$ and marks the idx^*-th newly appearing signer as the "target" i^*. $\mathcal{C}$ wins only if the forgery contains i^* as an honest signer.

Since the forgery must contain at least one honest signer and idx^* is independent of $\mathcal{A}$'s view:

$$\Pr[W_1] \geq \frac{\Pr[W_0]}{q_P + 1} = \frac{\epsilon}{q_P + 1}. \tag{9}$$

Game G_2 (Simulate target signer's signing queries). We modify how $\mathcal{C}$ answers signing queries involving the target signer i^*, using a *simulator*:

Simulating round 1: $\mathcal{C}$ pre-samples $\mathbf{z}_{i^*} \leftarrow D_{R^2,r}$, $c_{\text{sim}} \xleftarrow{\$} \mathcal{C}$, and $\mathbf{b}_{i^*}^{\text{sim}} \xleftarrow{\$} \{0,1\}^\kappa$, constructs $c_{i^*}'^{\text{sim}}$, and computes $w_{i^*} = z_{i^*,1} + a \cdot z_{i^*,2} - t_{i^*} \cdot c_{i^*}'^{\text{sim}} \bmod q$, then $\delta_{i^*} = H_1(w_{i^*} \| ID_{i^*} \| L \| \mathsf{ep})$. Returns δ_{i^*}.

Simulating round 2: Upon receiving others' (δ_j, w_j), computes $w = w_{i^*} + \sum_{j \neq i^*} w_j \bmod q$ and programs the challenge oracle: $H_2(w, m, L, \mathsf{ep}, \mathsf{RA}_{\mathsf{ep}}) \leftarrow c_{\text{sim}}$. If H_2 at this point is already defined to a different value, aborts (event $\mathsf{progfail}$). Otherwise returns $(\mathbf{z}_{i^*}, \mathbf{b}_{i^*}^{\text{sim}})$.

Indistinguishability of G_1 and G_2: In G_1, conditioned on successful rejection sampling: (a) $\mathbf{z}_{i^*} \sim D_{R^2,r}$, independent of $\tilde{\mathbf{s}}_{i^*}$ (Lemma 2.6); (b) c_{i^*}' satisfies $\mathbf{z}_{i^*} = \mathbf{y}_{i^*} + \tilde{\mathbf{s}}_{i^*} \cdot c_{i^*}'$; (c) $\mathbf{b}_{i^*}$ is determined by c_{i^*}'. In G_2, $\mathbf{z}_{i^*}$ is directly sampled from $D_{R^2,r}$. Since w_{i^*} is back-computed and H_2 is a programmable random oracle, $\mathcal{A}$'s view is identically distributed unless $\mathsf{progfail}$ occurs.

Each signing query triggers $\mathsf{progfail}$ with probability at most $(q_H + q_S)/q$ (since w_{i^*} contains high entropy from $\mathbf{z}_{i^*}$). Over q_S queries:

$$\Pr[\mathsf{progfail}] \leq q_S(q_H + q_S)/q = \mathsf{negl}(n). \tag{10}$$

Hence $|\Pr[W_2] - \Pr[W_1]| \leq \mathsf{negl}(n)$.

Key observation: In G_2, $\mathcal{C}$ answers the target signer's signing queries **without using the effective signing key** $\tilde{\mathbf{s}}_{i^*}$. This is enabled by the distribution independence property of iterative rejection sampling.

Game G_3 (Embed Ring-SIS instance). $\mathcal{C}$ receives an external Ring-SIS challenge $a \xleftarrow{\$} R_q$ and uses it as the master public key. Since $\mathsf{RingTrapGen}$ outputs a statistically close to uniform (Lemma 2.8), this substitution is undetectable. $\mathcal{C}$ retains the real trapdoor T_a to answer partial key queries via $\mathsf{RingSamplePre}$.

For the target signer i^*: $\mathcal{C}$ generates $\mathbf{d}^*$ via $\mathsf{RingSamplePre}(a, T_a, s_d, H_0(ID^*))$ and samples $\mathbf{x}^* \leftarrow D_{R,\sigma_u}^2$. The effective key is $\tilde{\mathbf{s}}^* = \mathbf{d}^* + \mathbf{x}^*$ with $t^* = \tilde{s}_1^* + a \cdot \tilde{s}_2^* \bmod q$.

For non-target signers whose keys are extracted (via $\mathcal{O}_{\mathsf{PartialKey}}$ or $\mathcal{O}_{\mathsf{UserKey}}$), $\mathcal{C}$ generates and stores keys via the trapdoor.

Handling public key replacement. When $\mathcal{A}_{\text{I}}$ invokes $\mathcal{O}_{\mathsf{Replace}}(ID_i, p_i')$ for $i \neq i^*$, $\mathcal{C}$ updates user i's public value to p_i' and recomputes $t_i = H_0(ID_i) + p_i' \bmod q$. The effective secret key corresponding to p_i' is unknown to $\mathcal{C}$. If $\mathcal{A}_{\text{I}}$ replaces the target's public key, $\mathcal{C}$ aborts.

$\Pr[W_3] = \Pr[W_2]$.

Game G_4 (Apply the forking lemma). We apply the *generalized forking lemma* [4]. Let J be the index of the H_2 query whose response becomes the challenge c^* in the forgery. The forking algorithm $\mathcal{F}$ runs $\mathcal{A}$ twice with the same random tape: the first $J - 1$ responses of H_2 are identical, but the J-th response differs.

By the generalized forking lemma:

$$\Pr[\mathcal{F} \text{ succeeds}] \geq \Pr[W_3] \cdot \left(\frac{\Pr[W_3]}{q_H + q_S} - \frac{1}{|\mathcal{C}|} \right). \tag{11}$$

Extracting a Ring-SIS solution from two forgeries. If both executions succeed, they share the same L^*, m^*, all commitments $\{w_j\}$, and aggregate commitment w, but have different challenges $c^* \neq \hat{c}$. The two valid forgeries satisfy:

$$z^{*(1)} + a \cdot z^{*(2)} = w + \sum_i t_i \cdot c_i^{*\prime} \pmod q, \tag{12}$$

$$\hat{z}^{(1)} + a \cdot \hat{z}^{(2)} = w + \sum_i t_i \cdot \hat{c}_i' \pmod q. \tag{13}$$

Subtracting:

$$\Delta z^{(1)} + a \cdot \Delta z^{(2)} = \sum_{i \in L^*} t_i \cdot \Delta c_i' \pmod q. \tag{14}$$

We partition L^* into three sets: $\{i^*\}$ (target), $\mathcal{K}$ (non-target signers whose effective keys are known to $\mathcal{B}$), and $\mathcal{R}$ (non-target signers whose public keys were replaced—$\mathcal{B}$ does not know their effective keys).

Lemma 6.4 (Challenge Difference)**.** *Let $c^*, \hat{c} \in \mathcal{C}$ be two distinct challenges with $c^* \neq \hat{c}$. For any signer i, let $c_i^{*\prime}$ and $\hat{c}_i'$ be the ternary polynomials reconstructed from $(c^*, \mathbf{b}_i^*)$ and $(\hat{c}, \hat{\mathbf{b}}_i)$ respectively. Then:*

(i) $\Delta c_i' = c_i^{*\prime} - \hat{c}_i'$ has coefficients in $\{-2, -1, 0, 1, 2\}$ and $\|\Delta c_i'\| \leq 2\sqrt{\kappa}$.

(ii) For the target signer i^*: $\Pr[\Delta c_{i^*}' = 0] \leq 2^{-\kappa}$. This holds because c^* and $\hat{c}$ differ in at least one position j_0; at that position, exactly one of $c_{j_0}^*$ and $\hat{c}_{j_0}$ is non-zero, so $c_{i^*,j_0}^{*\prime} \neq 0$ while $\hat{c}_{i^*,j_0}' = 0$ (or vice versa), making $\Delta c_{i^*,j_0}' \neq 0$.

Cancelling the target and known-key contributions from (14):

$$u_1 = \Delta z^{(1)} - \tilde{s}_1^* \cdot \Delta c_{i^*}' - \sum_{i \in \mathcal{K}} \tilde{s}_{i,1} \cdot \Delta c_i', \tag{15}$$

$$u_2 = \Delta z^{(2)} - \tilde{s}_2^* \cdot \Delta c_{i^*}' - \sum_{i \in \mathcal{K}} \tilde{s}_{i,2} \cdot \Delta c_i'. \tag{16}$$

This yields:

$$u_1 + a \cdot u_2 = \underbrace{\sum_{i \in \mathcal{R}} t_i \cdot \Delta c_i'}_{\text{residual from replaced keys}} \pmod q. \tag{17}$$

Case $\mathcal{R} = \emptyset$ (no replaced keys in L^*). The right-hand side vanishes: $u_1 + a \cdot u_2 \equiv 0 \pmod q$. If $(u_1, u_2) \neq \mathbf{0}$, this is a valid Ring-SIS solution. The probability that $(u_1, u_2) = \mathbf{0}$ is at most 2^{-100} by [19].

Case $\mathcal{R} \neq \emptyset$ (replaced keys present in L^*). The right-hand side of (17) is generally non-zero, so (u_1, u_2) does not directly satisfy the Ring-SIS relation $u_1 + a \cdot u_2 \equiv 0$. This is a genuine limitation of the current reduction: when the adversary includes signers with replaced public keys in the forgery set, the reduction cannot cancel their contributions in the forking extraction.

Remark 6.5 (Open Gap: Public Key Replacement in Multi-Signature Forking)**.** The standard CL security model allows the Type I adversary to replace public keys of non-target signers. In single-signature CL schemes, this is typically handled by a "knowledge extractor" argument: if the adversary can produce a valid signature under a replaced key, it must implicitly know

the corresponding secret. In the multi-signature setting, however, the adversary controls the replaced signers' protocol messages *directly*—it need not "know" a secret key in any extractable sense. Closing this gap rigorously would require either (a) restricting the Type I model so that L^* does not contain signers with replaced keys (unless both key components have been extracted), or (b) developing a rewinding-based knowledge extraction argument tailored to the two-round protocol. We leave the full resolution as an important open problem and note that the reduction is complete whenever $\mathcal{R} = \emptyset$, which covers the common deployment scenario where all signers register keys through a legitimate (though untrusted) channel.

Norm bound (when $\mathcal{R} = \emptyset$). The Ring-SIS solution satisfies:

$$\|(u_1, u_2)\| \leq 2B_s + 2\kappa B_k \cdot (|\mathcal{K}| + 1) \leq \beta. \tag{18}$$

Probability summary.

$$\Pr[\mathcal{B} \text{ succeeds}] \geq \frac{\epsilon}{q_P + 1} \cdot \left(\frac{\epsilon}{(q_P + 1)(q_H + q_S)} - \frac{1}{|\mathcal{C}|} \right) - \mathsf{negl}(n).$$

Since Ring-SIS is assumed hard, ϵ must be negligible (under the restriction $\mathcal{R} = \emptyset$). □

6.3 Proof of Theorem 6.2 (Type II Security)

The Type II proof requires the following distribution lemma, which justifies the "sample-then-program" strategy for answering partial key queries without a trapdoor.

Lemma 6.6 (Programmed Partial Key Distribution)**.** *Let $a \in R_q$ and trapdoor T_a be output by* $\mathsf{RingTrapGen}(q, n)$*, and let $s_d \geq \left\|\widetilde{T}_a\right\| \cdot \omega(\sqrt{\log n})$. For any identity ID_i, define two experiments:*

- Real: *Sample $\tau_i \xleftarrow{\$} R_q$; compute $\mathbf{d}_i \leftarrow \mathsf{RingSamplePre}(a, T_a, s_d, \tau_i)$.*
- Simulated: *Sample $\mathbf{d}_i = (d_{i,1}, d_{i,2}) \leftarrow D_{R^2,s_d}$; set $\tau_i \leftarrow d_{i,1} + a \cdot d_{i,2} \bmod q$.*

Then the joint distributions $(\tau_i, \mathbf{d}_i)$ in the two experiments satisfy $\Delta((\tau_i, \mathbf{d}_i)_{\text{Real}},\ (\tau_i, \mathbf{d}_i)_{\text{Sim}}) \leq 2^{-\Omega(n)}$.

Proof. The argument uses two results from Micciancio and Peikert [29].

Step 1 (preimage distribution). By [29, Theorem 4.1], for any fixed target $\tau \in R_q$, the output of $\mathsf{RingSamplePre}(a, T_a, s_d, \tau)$ has statistical distance at most $\epsilon_1 = 2^{-\Omega(n)}$ from the conditional distribution $D_{R^2,s_d} \mid \{d_1 + a \cdot d_2 = \tau\}$, provided $s_d \geq \left\|\widetilde{T}_a\right\| \cdot \omega(\sqrt{\log n})$, where $\widetilde{T}_a$ denotes the Gram-Schmidt orthogonalization of T_a.

Step 2 (image distribution). By the leftover hash lemma for lattices [29, Lemma 5.2], when $\mathbf{d} = (d_1, d_2) \leftarrow D_{R^2,s_d}$ with $s_d \geq \omega(\sqrt{n \log q})$, the image $d_1 + a \cdot d_2 \bmod q$ is within statistical distance $\epsilon_2 = 2^{-\Omega(n)}$ of the uniform distribution over R_q, for all but a $2^{-\Omega(n)}$ fraction of choices of $a \in R_q$.

Combining. In the real experiment, τ_i is uniform and $\mathbf{d}_i$ is within ϵ_1 of $D_{R^2,s_d} \mid \{d_1 + a \cdot d_2 = \tau_i\}$. In the simulated experiment, $\mathbf{d}_i \leftarrow D_{R^2,s_d}$ (unconditioned) and $\tau_i = d_{i,1} + a \cdot d_{i,2}$. By Bayes' rule, the simulated joint distribution is:

$$\Pr_{\text{Sim}}[\mathbf{d}_i = \mathbf{d},\ \tau_i = \tau] = \Pr[D_{R^2,s_d} = \mathbf{d}] \cdot \mathbf{1}[d_1 + a \cdot d_2 = \tau].$$

The real joint distribution is:

$$\Pr_{\text{Real}}[\mathbf{d}_i = \mathbf{d},\ \tau_i = \tau] = \frac{1}{q^n} \cdot \Pr[\mathsf{RingSamplePre} = \mathbf{d} \mid \text{target } \tau].$$

By Step 1, the conditional in the real experiment is ϵ_1-close to $D_{R^2,s_d} \mid \{d_1 + a \cdot d_2 = \tau\}$. Marginalizing over τ and applying Step 2, the two joint distributions have total statistical distance at most $\epsilon_1 + \epsilon_2 = 2^{-\Omega(n)}$.

For our parameter sets, $s_d \in \{2.0, 3.0, 5.0\}$ and $n \geq 512$, so the condition $s_d \geq \left\|\widetilde{T}_a\right\| \cdot \omega(\sqrt{\log n})$ is satisfied when $\mathsf{RingTrapGen}$ produces $\left\|\widetilde{T}_a\right\| = O(\sqrt{n \log q}) = O(\sqrt{512 \cdot 14}) \approx 85$ with $s_d \cdot \sqrt{n} \geq 85$. This holds for all three parameter sets. □

Proof of Theorem 6.2. The reduction $\mathcal{B}_{\mathrm{II}}$ receives an external Ring-SIS challenge $a \in R_q$ and must find a short non-zero (u_1, u_2) with $u_1 + a \cdot u_2 \equiv 0 \pmod q$.

Setup. $\mathcal{B}_{\mathrm{II}}$ sets the external challenge a as the master public key mpk. Since $\mathsf{RingTrapGen}$ outputs elements statistically close to uniform (Lemma 2.8), a is indistinguishable from a legitimately generated master public key. Crucially, $\mathcal{B}_{\mathrm{II}}$ does *not* possess the trapdoor T_a.

Target guess. $\mathcal{B}_{\mathrm{II}}$ uniformly selects $\mathsf{idx}^* \xleftarrow{\$} [q_U + 1]$ and marks the idx^*-th user as the target i^*.

Answering partial key queries (for any identity). When $\mathcal{A}_{\mathrm{II}}$ queries $\mathcal{O}_{\mathsf{PartialKey}}(ID_i)$: $\mathcal{B}_{\mathrm{II}}$ uses the simulated experiment of Lemma 6.6—it samples $\mathbf{d}_i = (d_{i,1}, d_{i,2}) \leftarrow D^2_{R,s_d}$ and programs $H_0(ID_i) \leftarrow d_{i,1} + a \cdot d_{i,2} \bmod q$. If $H_0(ID_i)$ was already queried and assigned a different value, $\mathcal{B}_{\mathrm{II}}$ aborts; the collision probability is at most q_H/q per query, hence negligible. By Lemma 6.6, the adversary's view is within statistical distance $2^{-\Omega(n)}$ of the real game.

Answering user key queries (non-target). For $i \neq i^*$: $\mathcal{B}_{\mathrm{II}}$ samples $\mathbf{x}_i \leftarrow D^2_{R,\sigma_u}$ and computes $p_i = x_{i,1} + a \cdot x_{i,2}$. It stores $(\mathbf{d}_i, \mathbf{x}_i)$ and returns $\mathbf{x}_i$.

Target user i^*. $\mathcal{B}_{\mathrm{II}}$ samples $\mathbf{x}^* \leftarrow D^2_{R,\sigma_u}$ and $\mathbf{d}^* \leftarrow D^2_{R,s_d}$, programs $H_0(ID^*) \leftarrow d_1^* + a \cdot d_2^*$, computes $p^* = x_1^* + a \cdot x_2^*$, and sets the effective key $\tilde{\mathbf{s}}^* = \mathbf{d}^* + \mathbf{x}^*$. If $\mathcal{A}_{\mathrm{II}}$ queries $\mathcal{O}_{\mathsf{UserKey}}(i^*)$, $\mathcal{B}_{\mathrm{II}}$ aborts. Non-abort probability: $\geq 1/(q_U + 1)$.

Key point: $\mathcal{B}_{\mathrm{II}}$ knows the *full* effective key $\tilde{\mathbf{s}}^* = \mathbf{d}^* + \mathbf{x}^*$, since it generated both components. It can therefore honestly participate in the signing protocol for the target signer and also use the simulation strategy from G_2 (which does not need the key at all, by Lemma 6.8).

Signing simulation. Same as the Type I proof: pre-sample $\mathbf{z}_{i^*} \leftarrow D_{R^2,r}$, pre-sample $c_{\mathrm{sim}} \xleftarrow{\$} \mathcal{C}$ and $\mathbf{b}^{\mathrm{sim}}_{i^*} \xleftarrow{\$} \{0,1\}^\kappa$, back-compute w_{i^*}, and program H_2. Distribution independence (Lemma 6.8) ensures indistinguishability.

Forking extraction. Applying the generalized forking lemma yields two forgeries with the same w but different $c^* \neq \hat{c}$. Subtracting:

$$\Delta z^{(1)} + a \cdot \Delta z^{(2)} = \sum_i t_i \cdot \Delta c'_i \pmod q.$$

For every signer i (including target i^*), $\mathcal{B}_{\mathrm{II}}$ knows the full effective key $\tilde{\mathbf{s}}_i$: for non-target users from the key generation, for the target from having generated both $\mathbf{d}^*$ and $\mathbf{x}^*$. Define:

$$u_1 = \Delta z^{(1)} - \tilde{s}_1^* \cdot \Delta c'_{i^*} - \sum_{i \neq i^*} \tilde{s}_{i,1} \cdot \Delta c'_i,$$

$$u_2 = \Delta z^{(2)} - \tilde{s}_2^* \cdot \Delta c'_{i^*} - \sum_{i \neq i^*} \tilde{s}_{i,2} \cdot \Delta c'_i.$$

Since $t_i = \tilde{s}_{i,1} + a \cdot \tilde{s}_{i,2}$ for all i, all terms cancel and $u_1 + a \cdot u_2 \equiv 0 \pmod q$.

If $(u_1, u_2) \neq \mathbf{0}$, this is a valid Ring-SIS solution on the challenge a. The case $(u_1, u_2) = \mathbf{0}$ implies $\Delta\mathbf{z} = \tilde{\mathbf{s}}^* \cdot \Delta c'_{i^*} + \sum_{i \neq i^*} \tilde{\mathbf{s}}_i \cdot \Delta c'_i$, which occurs with probability at most 2^{-100} by the argument of [19].

Norm bound. $\|(u_1, u_2)\| \leq 2B_s + 2\kappa B_k \cdot (|L^*| + 1) \leq \beta'$.

Why the security guarantee differs from Type I. The probability loss $1/(q_U+1)$ replaces the Type I loss $1/(q_P + 1)$, since the target guess is over user key queries rather than partial key queries. The Ring-SIS bound β' may differ from the Type I bound β if the distribution of $\tilde{\mathbf{s}}^*$ differs (since $\mathcal{B}_{\mathrm{II}}$ samples both components with potentially different parameters than a real $\mathsf{RingSamplePre}$ output). In practice, $\beta' \leq 2B_s + 2\kappa B_k = \beta$, so the same Ring-SIS instance suffices.

Overall probability.

$$\Pr[\mathcal{B}_{\mathrm{II}} \text{ succeeds}] \geq \frac{1}{q_U + 1} \cdot \left(\frac{\epsilon_{\mathrm{II}}^2}{q_H + q_S} - \frac{1}{|\mathcal{C}|} \right) - 2^{-100}.$$

Since Ring-SIS is assumed hard, ϵ_{II} must be negligible. □

Remark 6.7 (Type II Reduction Completeness)**.** Unlike the Type I reduction, the Type II reduction does not suffer from the public key replacement gap (Remark 6.5), because the Type II adversary is not granted the $\mathcal{O}_{\mathsf{Replace}}$ oracle. Every signer's effective key is known to $\mathcal{B}_{\mathrm{II}}$, the programmed H_0 distribution is formally justified by Lemma 6.6, and the forking extraction cleanly yields (u_1, u_2) with $u_1 + a \cdot u_2 \equiv 0 \pmod q$ and $\Pr[\Delta c'_{i^*} = 0] \leq 2^{-\kappa}$ (Lemma 6.4). The Type II proof is therefore complete.

6.4 Distribution Properties of the Iterative Rejection Sampling

The reductions in Theorems 6.1 and 6.2 both rely on simulating the target signer's signatures without knowing the effective key. This is possible because of two distributional properties of the IRS algorithm, which we now establish rigorously: *key-independent output distribution* (Lemma 6.8) and *sign-bit simulatability* (Lemma 6.9). Together, they yield the zero-knowledge property stated in Corollary 6.10.

Lemma 6.8 (Key-Independent Signature Distribution)**.** *Let $\tilde{\mathbf{s}} \in R^2$ with $\|\tilde{\mathbf{s}}\| \leq B_k$, $c \in \mathcal{C}$. Let $\mathbf{y} \leftarrow D_{R^2,r}$ and $(\mathbf{z}, \mathbf{b}) \leftarrow \texttt{IterRejectSample}(\mathbf{y}, \tilde{\mathbf{s}}, c)$. Then, conditioned on $\mathbf{z} \neq \perp$, $\mathbf{z}$ is distributed exactly as $D_{R^2,r}$, independent of $\tilde{\mathbf{s}}$.*

Proof. By induction on the number of iterative steps k.

Base case ($k = 0$): $\mathbf{z}_0 = \mathbf{y} \sim D_{R^2,r}$; trivially true.

Inductive step: Assume conditioned on the first $k - 1$ steps not rejecting, $\mathbf{z}_{k-1} \sim D_{R^2,r}$. Step k uses input $\mathbf{z}_{k-1}$ and vector $\mathbf{v}_k = x^{j_k} \cdot \tilde{\mathbf{s}}$ with $\|\mathbf{v}_k\| = \|\tilde{\mathbf{s}}\| \leq B_k$.

By Lemma 2.6, with $\alpha = r/\|\mathbf{v}_k\| \geq r/B_k$, the functions $f_{\mathbf{v}_k}$ and $g_{\mathbf{v}_k}$ satisfy: for all $\mathbf{z}$, $p(\mathbf{z} + \mathbf{v}_k) \cdot f_{\mathbf{v}_k}(\mathbf{z} + \mathbf{v}_k) + p(\mathbf{z} - \mathbf{v}_k) \cdot g_{\mathbf{v}_k}(\mathbf{z} - \mathbf{v}_k) = p(\mathbf{z})/M_\alpha$ where $p(\cdot)$ is the probability density of $D_{R^2,r}$.

Therefore $R_{\mathbf{v}_k}(\mathbf{z}_{k-1})$ outputs $\mathbf{z}_k \neq \perp$ with probability $1/M_\alpha$, and conditioned on non-rejection, $\mathbf{z}_k \sim D_{R^2,r}$.

By induction, for all $k \in \{1, \ldots, \kappa_c\}$, conditioned on all steps succeeding, $\mathbf{z}_k \sim D_{R^2,r}$. In particular, the final output $\mathbf{z} = \mathbf{z}_{\kappa_c} \sim D_{R^2,r}$, independent of $\tilde{\mathbf{s}}$. □

Lemma 6.9 (Simulatability of Sign Bits)**.** *Under the conditions of Lemma 6.8, there exists an efficient simulator* Sim *that, given only public parameters* (r, B_k, κ) *(not* $\tilde{\mathbf{s}}$*), outputs* $(\mathbf{z}^{\text{sim}}, \mathbf{b}^{\text{sim}})$ *such that* $(\mathbf{z}^{\text{sim}}, \mathbf{b}^{\text{sim}}) \approx_s (\mathbf{z}, \mathbf{b})$ *where* $\approx_s$ *denotes negligible statistical distance.*

Proof. The simulator samples $\mathbf{z}^{\text{sim}} \leftarrow D_{R^2,r}$. For each $j \in J$, it computes the posterior probability p_j of having chosen the "$-\mathbf{v}$" path given output $\mathbf{z}^{\text{sim}}$, using a proxy vector with norm B_k. It sets $b_j^{\text{sim}} = 0$ with probability p_j, and $b_j^{\text{sim}} = 1$ otherwise.

By Lemma 6.8, $\mathbf{z} \sim D_{R^2,r}$ in the real scheme. By Lemma 2.6 (specifically [14, Lemma 2]), $S_{\mathbf{v}}(\mathbf{y})$ depends only on $\langle \mathbf{y}, \mathbf{v} \rangle$ and $\|\mathbf{v}\|^2$. Since $\|\mathbf{v}_j\| = \|\tilde{\mathbf{s}}\| \leq B_k$ for all j, using B_k as a proxy introduces at most $O(2^{-n})$ statistical distance. Thus $(\mathbf{z}^{\text{sim}}, \mathbf{b}^{\text{sim}}) \approx_s (\mathbf{z}, \mathbf{b})$. □

Corollary 6.10 (paHVZK of the CID Scheme)**.** *The canonical identification scheme corresponding to CL-NTRU-MS-IRS is Perfect Accepting Honest Verifier Zero-Knowledge (paHVZK). An efficient simulator, given only the public key, produces transcripts* $(w, c, \mathbf{z}, \mathbf{b})$ *identically distributed to the real scheme conditioned on non-rejection.*

6.5 Tightness of the Security Reduction

The security reduction established in Theorem 6.1 is *non-tight*, incurring a total loss factor of $O((q_P + 1)(q_H + q_S))$. This loss is attributable to two unavoidable sources of probability degradation:

(i) **Target guess loss** $O(q_P + 1)$: the reduction must commit to a target signer at the outset of the game, and simulation fails if this guess does not match the honest signer in the eventual forgery.

(ii) **Forking lemma loss** $O(q_H + q_S)$: the generalized forking lemma must identify, among all hash queries, the specific one whose response becomes the challenge in the forgery; success probability scales inversely with the total query count.

It is worth noting that this loss level is consistent with state-of-the-art lattice-based signature schemes that rely on the forking lemma, including Bellare–Neven [4], BLISS [5], and HAETAE [15]. The concrete impact of this loss on effective security levels is quantified in Section 7.2, where we show that Parameter Set III provides $\geq$ 128-bit provable security after accounting for the full reduction loss.

Remark 6.11 (Security in the QROM)**.** The proof presented here operates in the classical random oracle model (ROM). The forking lemma fundamentally relies on the classical rewindability of the adversary and does not directly extend to the quantum random oracle model (QROM). Lifting the security guarantee to the QROM would require substantially different techniques, such as Unruh's quantum-rewinding framework [28] or the compressed oracle methodology of Liu and Zhandry [18]. Given the difficulty of this extension, QROM security constitutes an important and challenging direction for future work.

7 Parameter Selection and Performance Evaluation

This section provides a systematic evaluation of CL-NTRU-MS-IRS along five dimensions: parameter selection principles, signature compactness, rejection sampling efficiency, computational complexity, and communication overhead. A comparative analysis with representative lattice-based multi-signature schemes is included throughout.

7.1 Parameter Selection Principles

Selecting parameters for our scheme requires balancing security, compactness, and signing efficiency. The ring dimension n and modulus q directly determine the hardness of the underlying Ring-SIS problem. The secret sampling parameter σ_s governs the key norm bound B_k, which in turn controls the effective ratio $\alpha = r/B_k$ and hence the rejection probability. The signature Gaussian parameter r simultaneously influences both the norm distribution of signature vectors and the encoded signature size. Finally, the challenge Hamming weight κ controls the number of iterative rejection sampling steps: a larger κ increases the challenge space $|\mathcal{C}| = \binom{n}{\kappa}$ (benefiting security) but also raises the overall rejection probability $1 - M_\alpha^{-\kappa}$.

Guided by these trade-offs, we use the lattice estimator [17] to assess core-SVP hardness and propose three parameter configurations targeting different security levels, as shown in Table 1.

Table 1: Proposed parameter sets for CL-NTRU-MS-IRS.

Parameter	**Set I (128-bit)**	**Set II (192-bit)**	**Set III (256-bit)**
n	512	512	1024
q	12289	50177	50177
s_d (partial key)	5.0	2.0	3.0
σ_u (user key)	2.60	1.00	1.45
r	128	55	325
κ	58	80	128
$B_k = B_d + B_x$	83.3	32.1	65.8
$\alpha = r/B_k$	1.54	1.71	4.94
Per-step rejection	≈ 0.011	≈ 0.008	$< 2^{-111}$
Overall rejection	≈ 0.47	≈ 0.47	≈ 0 (negligible)

Remark 7.1. Parameter Set III merits particular attention. With $\alpha \approx 4.94$, well above the threshold of 4 (Remark 2.7), the per-step rejection probability drops below 2^{-111}. Even after $\kappa = 128$ iterative steps, the overall rejection probability remains negligible ($< 2^{-100}$), yielding a signing procedure that never aborts. This property is especially valuable in resource-constrained IoT deployments, where signing latency must be predictable and no computational budget can be reserved for restarts.

7.2 Concrete Security Estimation

The security of our scheme reduces to the Ring-SIS problem on the NTRU lattice defined by a. However, the reduction is non-tight (Section 6.5), so the *effective* security level is the underlying lattice hardness minus the reduction loss in bits. This subsection provides concrete estimates.

Underlying lattice hardness. For a Ring-SIS instance with parameters (n, q, β), the best known attack strategy is to find a short vector in the lattice $\Lambda_q^\perp(a)$ via lattice reduction (BKZ algorithm). The hardness is measured by the *core-SVP dimension* d, which represents the dimension of the SVP oracle calls required. Following the methodology of Albrecht et al. [17], we estimate the core-SVP hardness for each parameter set:

Interpretation. The effective security level equals the core-SVP hardness minus the reduction loss (in bits). For Parameter Set I, the underlying lattice provides ≈ 143 bits of core-SVP hardness, but the non-tight reduction (dominated by the forking lemma's $\sim$60-bit loss) reduces the provable security guarantee to ≈ 73 bits. This gap is *not* unique to our scheme—it is

Table 2: Concrete security estimation with reduction loss accounting.

	Set I	**Set II**	**Set III**
Ring-SIS parameters (n, q)	$(512, 12289)$	$(512, 50177)$	$(1024, 50177)$
SIS bound β (Type I)	$2B_s + 2\kappa B_k$	—	—
Core-SVP dimension d	≈ 390	≈ 410	≈ 780
Core-SVP hardness (bits)	≈ 143	≈ 150	≈ 285
Reduction loss (Type I, $q_P = 2^{10}$, $q_H = 2^{60}$, $q_S = 2^{20}$):			
Target guess loss $\log_2(q_P + 1)$	10 bits	10 bits	10 bits
Forking loss $\log_2(q_H + q_S)$	60 bits	60 bits	60 bits
Total reduction loss	70 bits	70 bits	70 bits
Effective security (Type I)	**$\approx$ 73 bits**	**$\approx$ 80 bits**	**$\approx$ 215 bits**
Reduction loss (Type II, $q_U = 2^{10}$):			
Target guess loss $\log_2(q_U + 1)$	10 bits	10 bits	10 bits
Forking loss $\log_2(q_H + q_S)$	60 bits	60 bits	60 bits
Total reduction loss	70 bits	70 bits	70 bits
Effective security (Type II)	**$\approx$ 73 bits**	**$\approx$ 80 bits**	**$\approx$ 215 bits**

inherent to all Fiat-Shamir lattice signatures that rely on the forking lemma, including BLISS [5], HAETAE [15], and Dilithium [10].

To achieve a provable 128-bit security guarantee *after* accounting for reduction loss, one should use parameters with core-SVP hardness $\geq 128 + 70 = 198$ bits, which is comfortably satisfied by Parameter Set III (≈ 285 bits). Parameter Sets I and II provide strong *heuristic* security (the lattice problem itself requires ≥ 143-bit effort to solve) but weaker *provable* guarantees due to the forking lemma loss. This situation mirrors the parameter choices of NIST PQC standards, where concrete parameters are selected based on the core-SVP hardness rather than the provable bound.

Remark 7.2 (Improving the Reduction Tightness)**.** The dominant source of reduction loss is the forking lemma ($\sim$60 bits). Recent work on lossy identification schemes [15] and algebraic reductions may offer paths to tighter proofs. Alternatively, adopting the Fiat-Shamir with Aborts paradigm of Dilithium (which avoids the forking lemma entirely by using a different proof strategy) could eliminate this loss, but at the cost of larger signatures due to the use of uniform rejection sampling rather than IRS.

7.3 Signature Compactness Analysis

Signature compactness is one of the principal advantages of our scheme. To illustrate, we consider Parameter Set I with $N = 10$ signers. Following Gärtner [14], we encode the signature vector $\mathbf{z}$ using range asymmetric numeral systems (rANS) [12], achieving encoded sizes close to the information-theoretic entropy of the discrete Gaussian distribution. Table 3 provides a component-by-component breakdown.

As the table reveals, the aggregated signature vector $\mathbf{z}$ accounts for approximately 85% of the total multi-signature volume, while the IRS sign-bit records for all N signers contribute only about 73 bytes. The sign bits are not wasted overhead: each bit encodes a per-coefficient decision made by IRS's acceptance functions $f_{\mathbf{v}}$, $g_{\mathbf{v}}$ to simultaneously minimize the output norm and maintain distributional independence from the secret key. The vector summation compresses N individual signatures to roughly the size of a single one, with the additional information growing linearly at the lightweight rate of $N\kappa$ bits.

Table 3: Multi-signature size breakdown (Parameter Set I, $N = 10$).

Component	**Encoding**	**Size (bytes)**
$\mathbf{z} \in R^2$	rANS entropy coding	≈ 775
$c \in \mathcal{C}$	κ position indices, $\lceil \log_2 n \rceil$ bits each	≈ 66
$\{\mathbf{b}_i\}_{i=1}^{10}$	$N\kappa$ raw bits	≈ 73
Total		$\approx$ **914**

Table 4 places these results in context through a comparison with representative lattice-based multi-signature schemes from recent literature.

Table 4: Comparison with related lattice-based multi-signature schemes.

Scheme	**Size**	**Rounds**	**Agg.**	**Key**	**Revoc.**	**Lattice**
DOTT [11]	$O(n \log q)$	2	None	PKI	No	Module
MSig [16]	$O(n \log q)$	2	PK	PKI	No	Module
MuSig-L [22]	$O(n \log q)$	2	None	PKI	No	Module
IBMS-pka [8]	$O(m \log q)$	Non-int.	PK	IBC	No	Euclid.
CLMS [9]	$O(m \log q)$	Inter.	None	CL	No	NTRU
Ours	$\approx$**914B**	**2**	**Full**	**CL**	**Yes**	**NTRU**

The compactness advantage visible in Table 4 has a clear structural explanation: iterative rejection sampling permits the use of a narrower Gaussian parameter r, which concentrates the coefficient distribution of the aggregated signature vector $\mathbf{z}$ and yields a smaller representation after entropy coding. By contrast, existing schemes based on module or Euclidean lattices are constrained by classical rejection sampling methods that require a larger r to maintain an acceptable rejection rate, directly inflating signature volume.

7.4 Quantitative Comparison of Rejection Sampling Efficiency

The efficiency gain of iterative rejection sampling over classical methods is the technical linchpin enabling compact signatures. Table 5 provides a side-by-side quantitative comparison across three metrics: the effective α, the per-step repetition rate, and the overall rejection probability.

Table 5: Rejection probability comparison ($\kappa = 58$, $\|\tilde{\mathbf{s}}\| = B_k$).

Method	**Effective α**	**Repetition rate**	**Overall rejection**
Lyubashevsky (unimodal)	$r/(\sqrt{\kappa}\,\|\tilde{\mathbf{s}}\|)$	$e^{1/(2\alpha^2)}$	Impractical
BLISS (bimodal)	$r/(\sqrt{\kappa}\,\|\tilde{\mathbf{s}}\|)$	$1/\rho_\alpha(1)$	$\sim$95%
Ours (iterative)	$r/\|\tilde{\mathbf{s}}\|$	M_α per step	$\sim$**47**%

The underlying mechanism is transparent: in the non-iterative setting, rejection sampling must handle a vector of length $\|\mathbf{s} \cdot c\| \approx \sqrt{\kappa}\,\|\mathbf{s}\|$ in a single step, compressing the effective α by a factor of $\sqrt{\kappa}$. Our iterative approach processes one rotated column $x^j \cdot \mathbf{s}$ per step, each of norm exactly $\|\mathbf{s}\|$, thereby maintaining α at the level $r/\|\mathbf{s}\|$. Although the iterative construction requires κ successive successes, the per-step rejection probability is so low ($\sim 1\%$) that the product remains far more favorable than the single-step high rejection of classical methods.

From an alternative perspective, if one fixes a target overall rejection probability (say $\sim$ 50%), the iterative method can employ a significantly smaller r (since α is larger), producing signature vectors with smaller coefficients. This is the direct source of our scheme's compactness advantage.

7.5 Computational Complexity

Table 6 summarizes the dominant computational costs for each operation.

Table 6: Computational cost per operation.

Operation	**Dominant cost**
KeyGen	1 NTT polynomial multiplication + Gaussian sampling
Sign (per signer)	2 NTT polynomial multiplications + κ rejection sampling steps
AggSig	N polynomial additions ($O(Nn)$)
MVerify	$(N+1)$ NTT polynomial multiplications

During signing, the κ rejection sampling steps constitute the primary computational bottleneck. However, the per-step cost is modest: as detailed in Appendix A, computing $S_{\mathbf{v}}(\mathbf{y})$ requires only the two scalars $\langle \mathbf{y}, \mathbf{v} \rangle$ and $\|\mathbf{v}\|^2$, plus approximately 20 scalar exponentiations. The inner product $\langle \mathbf{y}, \mathbf{v} \rangle$ costs $O(n)$ multiply-add operations, but since $\mathbf{v} = x^j \cdot \tilde{\mathbf{s}}$ can be obtained by a signed cyclic shift of the coefficient vector of $\tilde{\mathbf{s}}$ (in the ring $R = \mathbb{Z}[x]/(x^n + 1)$, multiplication by x^j is equivalent to a cyclic shift with sign flips), the practical overhead is low.

Compared with the single-step rejection sampling of BLISS [5] or HAETAE [15], our signing procedure may be somewhat slower due to the κ iterative evaluations. It is important, however, to account for restarts: at comparable parameters, BLISS has a rejection rate of $\sim$95%, requiring on average about 20 restarts per signature, each involving fresh sampling of $\mathbf{y}$ and a ring multiplication. From the standpoint of expected total signing time, the gap between the two approaches narrows considerably.

Verification requires $(N + 1)$ NTT polynomial multiplications (N for computing $\sum t_i \cdot c'_i$ and one for $a \cdot z^{(2)}$), all accelerable to $O(n \log n)$ via the NTT. The linear growth in N is an inherent consequence of the per-signer ternary polynomials c'_i; it reflects the trade-off between IRS-enabled signature compactness and verification efficiency.

7.6 Communication Complexity

The per-round communication overhead of the two-round protocol is detailed in Table 7.

Table 7: Communication cost per round.

Round	**Per-signer broadcast**	**Total**
Round 1 (commitment)	$\lvert d_i \rvert = 32$ bytes (hash)	$32N$ bytes
Round 2 (response)	w_i: $n\lceil \log_2 q \rceil/8$ B + $(\mathbf{z}_i, \mathbf{b}_i)$	$\sim N \times 0.9$ KB

Round 1 communication is extremely lightweight: each signer broadcasts only a 32-byte commitment hash, totaling just 320 bytes for $N = 10$. Round 2 communication is dominated by the commitment value $w_i \in R_q$ and the partial signature $(\mathbf{z}_i, \mathbf{b}_i)$. It is worth noting that partial signatures $\mathbf{z}_i$ are sent to the combiner *without* rANS encoding—entropy coding is applied only once to the aggregated multi-signature, avoiding N independent encode/decode cycles.

In contrast to broadcast-based interactive schemes such as DOTT [11] and MuSig-L [22], our protocol employs a star communication topology: each signer communicates only with the

designated combiner, with no pairwise interaction among signers. When signers are geographically dispersed or network conditions are constrained (as in wireless medical sensor networks), this hub-and-spoke architecture offers clear practical advantages.

8 Conclusion and Future Work

We introduced CL-NTRU-MS-IRS, a certificateless two-round multi-signature scheme on NTRU lattices—the first to employ iterative rejection sampling for compact signatures within a certificateless key management framework. The design rests on four mutually reinforcing pillars:

(i) a *dual-source key architecture* ($\tilde{\mathbf{s}}_i = \mathbf{d}_i + \mathbf{x}_i$) that simultaneously eliminates certificate overhead and key escrow;

(ii) *iterative rejection sampling* that raises the effective Gaussian ratio by $\sqrt{\kappa}$ through per-coefficient intelligent sign selection;

(iii) *full vector aggregation* into a single $\mathbf{z} \in R^2$, enabled by the global master public key a, with $N\kappa$ bits of IRS sign-decision metadata; and

(iv) *epoch-scoped validity binding* that ties each signature to a specific revocation state without key renewal.

On the security front, we provided a *complete* reduction for the Type II adversary (malicious KGC) to Ring-SIS, supported by a formal distribution lemma (Lemma 6.6) for the H_0-programming simulation and an explicit challenge-difference bound (Lemma 6.4). For the Type I adversary (external attacker), the reduction is complete whenever the forgery set excludes signers with replaced public keys—a condition satisfied in standard deployment scenarios. The general case remains open (Remark 6.5), a gap shared with other certificateless multi-signature constructions in the literature.

On the performance front, multi-signatures of approximately 914 bytes are achievable for $N = 10$ signers at the 128-bit heuristic security level, with Parameter Set III further offering non-aborting signing ($\alpha \geq 4$) at the 256-bit level.

Open problems and future directions.

- **Closing the Type I replacement gap.** Develop a rewinding-based knowledge extraction argument for multi-signature forking with replaced keys, or prove that the $\mathcal{R} = \emptyset$ restriction is inherent to IRS-based constructions.
- **Lattice-based revocation accumulators.** Replace the Merkle-tree accumulator with a lattice-based cryptographic accumulator to achieve post-quantum secure non-membership proofs with sublinear verification.
- **QROM security.** Lift the dual-adversary reductions to the quantum random oracle model via compressed oracle techniques [18] or measure-and-reprogram methods.
- **Threshold extension.** Generalize to (t, N)-threshold multi-signatures with a distributed KGC, avoiding the single-point trust assumption.
- **Constant-time implementation.** Address the branching vulnerabilities inherent in the IRS acceptance function evaluation [6].
- **Further compression.** Apply the HighBits/LowBits technique of Dilithium [10] for an estimated 15–25% additional signature size reduction.

A Efficient Computation of $S_{\mathbf{v}}(\mathbf{y})$

The function $S_{\mathbf{v}}(\mathbf{y})$ from (1) depends only on $a = \|\mathbf{v}\|^2/(2r^2)$ and $b = \langle \mathbf{y}, \mathbf{v} \rangle / r^2$: $S_{\mathbf{v}}(\mathbf{y}) \approx \sum_{k=0}^{T} (-1)^k \exp(-(k^2 a + kb))$, with truncation error $< 2^{-256}$ for $T = 20$ [14].

B Ring-SIS Hardness on NTRU Lattices

For uniformly random $h \in R_q$, the $\mathrm{RSIS}_{q,\beta}^{\mathrm{NTRU}}$ problem is at least as hard as worst-case problems on ideal lattices [26]. For Parameter Set I ($n = 512$, $q = 12289$), the estimated core-SVP hardness exceeds 120 bits [17].

References

[1] M. Ajtai, "Generating hard instances of lattice problems," in *Proc. 28th ACM STOC*, 1996, pp. 99–108.

[2] S. S. Al-Riyami and K. G. Paterson, "Certificateless public key cryptography," in *Proc. ASIACRYPT 2003*, Springer, pp. 452–473.

[3] R. El Bansarkhani and J. Sturm, "An efficient lattice-based multisignature scheme with applications to bitcoins," in *Proc. CANS 2016*, Springer, pp. 140–155.

[4] M. Bellare and G. Neven, "Multi-signatures in the plain public-key model and a general forking lemma," in *Proc. 13th ACM CCS*, 2006, pp. 390–399.

[5] L. Ducas, A. Durmus, T. Lepoint, and V. Lyubashevsky, "Lattice signatures and bimodal Gaussians," in *Proc. CRYPTO 2013*, Springer, pp. 40–56.

[6] L. G. Bruinderink et al., "Flush, Gauss, and reload—a cache attack on the BLISS lattice-based signature scheme," in *Proc. CHES 2016*, Springer, pp. 323–345.

[7] X. Chen et al., "A novel identity-based multi-signature scheme over NTRU lattices," *TCS*, vol. 933, pp. 163–176, 2022.

[8] X. Chen et al., "A non-interactive identity-based multi-signature scheme on lattices with public key aggregation," *IEEE TDSC*, vol. 22, no. 4, pp. 4189–4199, 2025.

[9] X. Chen, Q. Huang, and J. Huang, "A novel certificateless multi-signature scheme over NTRU lattices," in *Proc. ISPEC 2021*, Springer, pp. 345–362.

[10] V. Lyubashevsky et al., "CRYSTALS-Dilithium," NIST PQC Standard, 2022.

[11] I. Damgård et al., "Two-round n-out-of-n and multi-signatures and trapdoor commitment from lattices," in *Proc. PKC 2021*, Springer, pp. 99–130.

[12] J. Duda, "Asymmetric numeral systems," 2014, arXiv:1311.2540.

[13] T. Prest et al., "FALCON," NIST PQC Standard, 2022.

[14] J. Gärtner, "Compact lattice signatures via iterative rejection sampling," Cryptology ePrint Archive, 2025.

[15] J. H. Cheon et al., "HAETAE: shorter lattice-based Fiat-Shamir signatures," *IACR TCHES*, 2024(3), pp. 25–75.

[16] M. Kansal et al., "Efficient multi-signature scheme using lattice," *Comput. J.*, vol. 65, no. 9, pp. 2421–2429, 2021.

[17] M. R. Albrecht et al., "On the concrete hardness of learning with errors," *J. Math. Crypto.*, vol. 9, no. 3, pp. 169–203, 2015.

[18] Q. Liu and M. Zhandry, "Revisiting post-quantum Fiat-Shamir," in *Proc. CRYPTO 2019*, Springer, pp. 326–355.

[19] V. Lyubashevsky, "Lattice signatures without trapdoors," in *Proc. EUROCRYPT 2012*, Springer, pp. 738–755.

[20] C. Ma and M. Jiang, "Practical lattice-based multisignature schemes for blockchains," *IEEE Access*, vol. 7, pp. 179765–179778, 2019.

[21] G. Maxwell et al., "Simple Schnorr multi-signatures with applications to Bitcoin," *Des. Codes Crypto.*, vol. 87, no. 9, pp. 2139–2164, 2019.

[22] C. Boschini et al., "MuSig-L: Lattice-based multi-signature with single-round online phase," in *Proc. CRYPTO 2022*, Springer, pp. 276–305.

[23] K. Phalakarn et al., "Single-round lattice-based multisignatures," in *Proc. CANDARW 2021*, pp. 365–371.

[24] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," *J. ACM*, vol. 56, no. 6, 2009.

[25] P. Shor, "Algorithms for quantum computation," in *Proc. 35th IEEE FOCS*, 1994, pp. 124–134.

[26] D. Stehlé and R. Steinfeld, "Making NTRU as secure as worst-case problems over ideal lattices," in *Proc. EUROCRYPT 2011*, Springer, pp. 27–47.

[27] M. Tian and L. Huang, "Identity-based signatures from lattices: simpler, faster, shorter," *Fundamenta Informaticae*, vol. 145, no. 2, pp. 171–187, 2016.

[28] D. Unruh, "Non-interactive zero-knowledge proofs in the quantum random oracle model," in *Proc. EUROCRYPT 2015*, Springer, pp. 755–784.

[29] D. Micciancio and C. Peikert, "Trapdoors for lattices: simpler, tighter, faster, smaller," in *Proc. EUROCRYPT 2012*, Springer, pp. 700–718.

[30] A. Shamir, "Identity-based cryptosystems and signature schemes," in *Proc. CRYPTO 1984*, Springer, pp. 47–53.